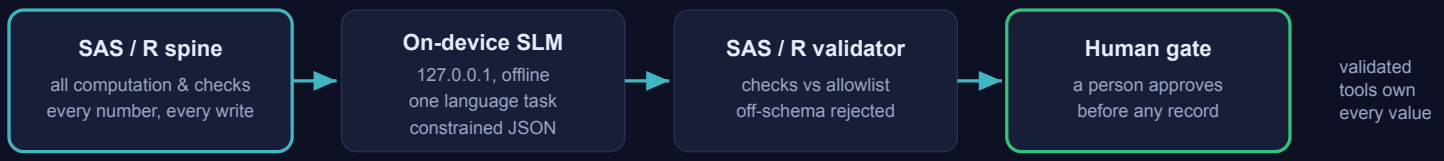
On-device Small Language Models + SAS/R — the 75-component stack

⚠ **PROPOSED — pending management approval, not in production today.** This describes a **proposed future-state** operating layer, not how we work now. Today's deliverables run through validated tools + our in-house team; every AI output is human-signed. New here? Start with the onboarding page.

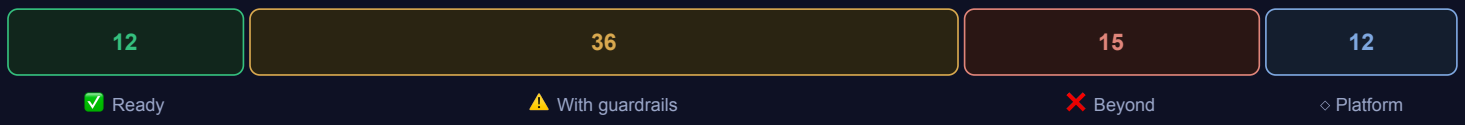
We took the **75-component hybrid trial-operations platform** and re-assessed **every component** for a **SMALL open-weight model running on-device** (a 1–9B quantized model on a workstation, fully offline) paired with **SAS/R**. The principle is strict: **SAS/R owns every number and check; the small model only classifies, extracts, routes, and drafts — always behind a human gate.**

12	36	15	12
✅ on-device ready	⚠ with guardrails	❌ beyond a small model	◇ platform / SAS·R

The one loop: SAS/R owns the numbers, the small offline model only helps with words



How the 75 components land for a small offline model



Only 12 are clean wins; most useful work sits in the guardrailed 36 (constrained output + RAG + validator + human gate); 15 genuinely need a bigger engine or stay deterministic SAS/R; 12 are infrastructure the offline posture makes simpler.

The whole stack in one picture: SAS/R owns every number and calls a small **offline** model for one narrow language task only — behind a validator and a human gate — and that is why just **12** of 75 components are clean wins, **36** work only with guardrails, and **15** are beyond a small model.

The honest headline. An on-device small model is the **most sovereign and cheapest** option — data physically never leaves the workstation, it runs on hardware you already own, and it is trivially frozen for reproducibility. But it is a **narrow language helper, not a reasoning engine**: the 12 clean wins and 36 guardrailed components are real, while the 15 marked *Beyond* genuinely need synthesis / long-context / nuance a small quantized model can't do reliably — those stay deterministic SAS/R or escalate to a larger model. The 12 *Platform* items aren't model tasks at all.

Validated tools (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS) still own every reported number and record — the "RED" governance is unchanged. What changes here is that the language helper is a *small, local, offline* model, so the data-sovereignty story is the simplest of any option — at the cost of model capability.

Why on-device SLM + SAS/R

This is the third variant of the same 75-component, early-phase clin-pharm trial-operations stack. The first ran on cloud Claude; the second on a big local GPU behind a model-agnostic gateway. This one asks a narrower question: how far do we get when the only model is a **small open-weight model running on hardware we already own**, fully offline, paired with the SAS/R we already validate against? The honest answer is: further than a skeptic expects on the language plumbing, and not as far as the marketing on anything that needs real reasoning. The division of labor is what makes it work.

> **The honest headline.** A small on-device model is a **constrained language helper, not a reasoning engine**. It can classify a short passage into a fixed set of buckets, pull a named field, route a queue, group near-duplicates, and draft one templated line for a human to confirm. It cannot be trusted with a number, a long synthesis, or a clinical judgement. So we hand it only the narrow language sub-task, behind a schema validator and a human gate — and **SAS/R owns the truth; the small model just helps with words.**

What "on-device SLM" actually means here

A 1–9B-parameter open-weight model — Qwen3 (0.6B–8B), Gemma 4 (E2B/E4B), Phi-4-mini (3.8B), IBM Granite 4.1 (3B/8B), Llama 3.3/3.2 (8B/3B/1B) — quantized to 4–8 bit (GGUF) and served *locally* by Ollama, llama.cpp, or LM Studio on an ordinary workstation or laptop, CPU or a single consumer GPU. (Ollama and LM Studio both wrap the llama.cpp inference engine; hard *grammar-constrained* decoding — GBNF in llama.cpp — is most directly controlled through llama.cpp / `llama-server`, which is what we standardize on for any call that must return schema-valid JSON.) No GPU farm, no cloud account, no telemetry, no network call leaving the box. SAS/R reaches it over `127.0.0.1` (PROC HTTP / httr2) for the language step only.

> **Licences are not uniform — check before you ship.** "Open-weight" does not mean "do anything." Granite 4.1, Qwen3 and Gemma 4 are Apache 2.0 (clean). But Llama 3.x ships under Meta's *Community Licence* (source-available, not OSI open source: a 700M-MAU cap and a "don't use outputs to train other models" clause), the earlier Gemma 2 and 3 shipped under Google's custom *Gemma Terms of Use* with a Prohibited Use Policy — though Gemma 4, from April 2026, moved to Apache 2.0, and Qwen likewise consolidated to uniform Apache 2.0 in Qwen3. For internal, non-redistributed trial-ops use these terms are generally workable, but the model choice is a licence decision QA/legal should sign off, not a detail.

The case for the small-model pairing

- **Maximum sovereignty — the simplest possible HIPAA story.** The data physically never leaves the workstation. There is no cloud egress to negotiate, no BAA to scope around an AI vendor, no EU-data-boundary question. The control is one sentence: *the box stays offline.*
- **No new hardware.** A 3B–9B quantized model runs on a workstation or laptop the team already has — a 3.8B model fits in a few GB of VRAM, and 4-bit 7B–9B models run on CPU. No GPU procurement, no server build-out as a precondition to start.
- **Negligible marginal cost.** After setup there is no per-token metering and no usage budget to police; the only cost is compute time and power. One workstation does serialize calls, so throughput — not price — is the practical limit.
- **Freezable for reproducibility — with one honest caveat.** We pin the model name + quantization + digest, the decoding settings (temperature 0, fixed seed), and the exact prompt; that bundle is an archivable, re-runnable artifact under change control. Re-run on the *same pinned build and hardware* and you get the same behavior. The honest caveat: temperature 0 + a fixed seed do **not** guarantee bitwise-identical tokens across *different*

GPUs/drivers or under batching/caching — CPU, single-threaded, fixed-build execution is the reproducible configuration we validate against. This is far closer to a GAMP-5 / 21 CFR Part 11 story than the cloud variant can offer (we control the binary and it never changes underneath us), but the regulatory guarantee comes from the *deterministic SAS/R validator and the human gate*, not from the model being perfectly deterministic.

- **No vendor or network dependency.** Nothing breaks because a model was deprecated, an API changed, or the link went down. The capability is on the box, and it stays the version we validated.

The capability ceiling — stated plainly

A small quantized model hallucinates *more* than a frontier model and is *more* sensitive to prompt and format. It degrades well before its advertised context window. The bigger-is-better instinct is correct: when the language work needs genuine reasoning, a 7B quantized model is not enough. So we never let it near these:

- **Anything numeric or quantitative** — math, counts, reconciliation, statistics. SAS/R owns *all* numbers, always. Reported and regulated values come only from validated engines (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS) — never the model.
- **Multi-document synthesis or long-context reasoning** — small models fall apart here first.
- **Nuanced clinical, safety, or regulatory narrative, judgement, or adjudication.**
- **Faithful long summaries; cross-correlating many signals.**

The division of labor — non-negotiable

SAS/R is the spine. It owns data access, every computation, check, reconciliation, and statistic, the scheduling and orchestration, the validator that checks the model's structured output against the allowed values, every write to a record of truth, and the audit log. The model is a sidecar SAS/R calls for one bounded language task, with JSON/grammar-constrained output and a human approving before anything touches a record.

SAS/R
data access · all computation, checks, stats · scheduling · the validator · every write · audit log

↓

On-device SLM (127.0.0.1)
narrow language task only — classify · extract · route · group · draft one line · constrained JSON out, temp 0, fixed seed

↓

SAS/R validator
checks the model's output against allowed values; rejects anything off-schema

↓

Human gate
a person confirms before it touches a record of truth — validated tools own every reported number

Where it is **reliably good** — and only *with* RAG grounding, constrained output, a SAS/R validator, and a narrow prompt — is short-text classification into fixed buckets, field extraction from a short passage, routing and severity tagging, near-duplicate grouping, short templated drafting grounded in retrieved text (a query line, a reminder email, a

candidate code for human confirmation), and yes/no/which-bucket calls over a small context. That is exactly the language seam across this trial-ops stack, and SAS/R does everything around it.

How the 75 components landed

We assigned every component exactly one verdict for the on-device-SLM + SAS/R variant.

Verdict	Count	What it means here
Ready	12	The language sub-task sits squarely in the reliable zone; SAS/R does all the deterministic work; low residual risk behind the human gate. A clean on-device win.
Guardrails	36	Feasible on-device *only* with tight scaffolding — constrained output + RAG + SAS/R validator + narrow scope + human gate — because the task edges toward longer context, more nuance, or higher stakes. Real residual hallucination/quality risk, named per component.
Beyond	15	The language work genuinely needs frontier-level reasoning, long-context synthesis, or nuanced narrative a small quantized model can't do reliably. Each says whether to keep it *deterministic SAS/R-only* (no model at all) or *escalate* to a larger local or cloud model — out of scope of "on-device SLM."
Platform	12	Pure infrastructure — gateway, egress controls, audit trail, approval queue, RBAC/secrets, model-freeze registry, the servers themselves — not an SLM language task. Noted is how the on-device variant simplifies it.

The shape of that distribution is the honest verdict on the variant. Only **12** components are clean on-device wins; **36** are feasible *but only behind real guardrails*; and **15** are simply Beyond a small model — kept deterministic in SAS/R or escalated. The remaining **12** are infrastructure, and this is where the offline posture pays off: a single offline local model means there is no cloud egress to firewall (the control collapses to "the box stays offline"), and the model-agnostic gateway collapses to "SAS/R calls one local endpoint." The plumbing gets simpler precisely because the sovereignty is absolute.

> **How to read the rest of this wiki.** Treat *Ready* as the place to start, *Guardrails* as buildable with the scaffolding spelled out and the residual risk owned, and *Beyond* as an honest "not with this model" — deterministic SAS/R or a bigger engine. Throughout, the same rule holds: validated tools and SAS/R own every reported and regulated value, the model only ever helps with words, and a human confirms before anything is real.

What a small model can and can't do

An on-device small model is a **narrow language helper, not a reasoning engine**. Knowing exactly where its useful work ends is the whole game: scope it to the tasks it does reliably, scaffold those tasks tightly, and keep everything else in SAS/R. Across the 75-component stack only **12** components land in the clean on-device zone; **36** are feasible only behind guardrails, **15** are genuinely beyond a small model, and **12** are pure platform plumbing. That distribution is the honest shape of the envelope: a small model earns its place on short, bounded, structured language sub-tasks and nowhere else.

Where a small model is safe — and where it is the wrong tool

✅ Safe — short, bounded, structured

- Short-text classification into fixed buckets
- Field / entity extraction from a short passage
- Routing & triage / severity tagging
- Near-duplicate / similarity grouping
- Short templated drafting grounded in retrieved text
- Yes / no / which-bucket over a small context

…only behind all four guardrails at once:

RAG grounding · constrained output · SAS/R validator · narrow prompt

❌ Never trust it with

- Anything numeric — SAS/R owns every number
- Multi-document synthesis / long-context reasoning
- Nuanced clinical / safety / regulatory judgement
- Faithful long summaries
- Cross-correlating many signals at once

This is the capability ceiling, not a tuning knob.

It fails fluently and confidently here — which is worse than failing loudly. Keep deterministic SAS/R, or escalate to a larger engine.

The envelope: a small model is safe only where the input is short, the output space is small and pre-defined, and a deterministic check can confirm the answer — and only behind RAG, constrained output, a validator, and a narrow prompt; the moment it must reason, synthesize, or touch a number it is the wrong tool.

The one rule that never bends. The small model never produces, checks, or reconciles a number. **SAS/R owns all computation, counts, reconciliation, and statistics**, and the validated engines — Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS — own every reported and regulated value. The model only classifies, extracts, routes, and drafts *words*, and only as a candidate that a human approves before it touches a record of truth.

What an on-device SLM actually is

An on-device small language model (SLM) is a **1–9B-parameter open-weight model**, quantized to roughly 4–8 bits and shipped as a **GGUF** file, served **locally** by a runtime such as Ollama, llama.cpp, or LM Studio on an ordinary workstation or laptop (CPU, or a single consumer GPU), **fully offline with no telemetry**. Representative families in mid-2026 include Qwen3 (0.6B–8B dense), Gemma 4 (E2B/E4B), Phi-4-mini (3.8B), IBM Granite 4.1 (3B/8B), and Llama 3.3/3.2 (8B/3B/1B). **"Open-weight" means the weights are downloadable — it does not mean OSI open-source, and licences differ:** Qwen3, Granite 4.1 and Gemma 4 ship under Apache 2.0, whereas Llama carries a custom community licence with acceptable-use restrictions. Confirm the licence for the specific model and version before any regulated use.

Reliable — but only with guardrails

These are the language sub-tasks a 1–9B quantized model does dependably, and only when wrapped in all four guardrails at once: **RAG grounding** (retrieve a short, relevant passage first), **constrained output** (JSON-schema or grammar-restricted), a **deterministic SAS/R validator** that checks every field against an allowlist, and a **narrow prompt**. Remove any one of the four and the reliability drops.

- **Short text classification** into a fixed set of buckets — e.g. labelling a finding by type or a document by TMF artifact.
- **Field / named-entity extraction** from a *short* passage — a date, an expiry, a site number, an owner pulled from one paragraph, not a whole document.
- **Routing & triage tagging** — which owner, queue, or severity bucket an item belongs in.
- **Near-duplicate / similarity grouping** — is this finding new, or the same as one we already raised?
- **Short, templated drafting grounded in retrieved text** — a query text, a reminder email, a one-line status note, or a candidate code for a human to confirm. The human, not the model, decides.
- **Yes / no / which-bucket decisions** over a small context.

The common thread: the input is short, the output space is small and pre-defined, and a deterministic check can confirm the answer is one of the allowed values. That is exactly where a small model is safe.

Do NOT trust a small model with

These are not deployment details to tune around — they are the capability ceiling. A small quantized model will fail on them in ways that look fluent and confident, which is worse than failing loudly.

- **Anything numeric or quantitative** — math, counts, reconciliation, statistics. SAS/R owns every number, always. The model is never in the number path.
- **Multi-document synthesis or long-context reasoning** — small models degrade well before their advertised context window, and cross-correlating many signals is exactly what they cannot hold.
- **Nuanced clinical, safety, or regulatory narrative, judgement, or adjudication** — the stakes and the nuance are both too high.
- **Faithful long summaries** — the longer the source and the longer the output, the more a small model invents or drops.

> **Small models hallucinate more than frontier models, and are more sensitive to prompt and format.** The bigger-is-better instinct is correct: when the language work needs real reasoning, a 7B quantized model is not enough. Those tasks stay deterministic SAS/R, or escalate to a larger local or cloud model — which is outside the on-device-SLM scope.

Division of labor

SAS/R owns data access, all computation, checks, reconciliation, and statistics, the orchestration and scheduling, the validator that checks the model's structured output against allowed values, every write, and the audit log. The SLM is a constrained language sidecar that SAS/R calls over a **local loopback endpoint** (PROC HTTP or httr2 to a `127.0.0.1` Ollama or llama.cpp server) for the language sub-task *only*, with schema-validated output and a **human gate** before anything reaches a record of truth. Reported and regulated numbers come only from validated engines (Pinnacle 21, Phoenix WinNonlin, EDC/CTMS) — never the SLM.

Quantization: the quality / footprint trade

An on-device model is shipped as a **GGUF** file at a chosen **quantization** — the weights are compressed from 16-bit to roughly 4–8 bits so the model fits in ordinary RAM and runs at usable speed. The trade is direct: lower bit-width means a smaller, faster model but degraded instruction-following and schema adherence.

- `Q4_K_M` — the common footprint choice; a 7–8B model is roughly **4–5 GB on disk and about 5–6 GB resident at runtime** (weights plus KV cache and overhead), so plan for materially more system RAM than the file size — on the order of 16 GB+ for comfortable CPU inference. It runs on CPU, just slowly; a consumer GPU is much faster. Good enough for many classification and tagging tasks *if* you validate it on yours.
- `Q8_0` — higher fidelity, larger footprint (roughly double `Q4_K_M`); closer to the full model's behaviour, worth it where schema adherence at `Q4` is shaky.

The rule: pick the **smallest model and lowest bit-width that still pass your validation set** for the specific task — never assume a quantization is fine because it works in a demo.

Why constrained output + a SAS/R validator is mandatory

A small model, left to free-form, will wrap its answer in prose, invent a category that was never offered, or drift the format between runs. Two mechanisms make it usable — and a human still closes the loop, because neither one certifies that the answer is *correct*:

Constrained output

SAS/R passes a JSON schema (Ollama `format`) or a GBNF grammar (llama.cpp), which the runtime compiles to a token mask — tokens that would break the grammar are forbidden at sampling, so the *shape* is guaranteed valid.

↓

SAS/R validator

SAS/R parses the JSON and checks every field against an allowlist / enum. Anything malformed or off-list is rejected and logged. This confirms the value is *in-bounds* — not that it is the *right* value.

↓

Human gate

A person reviews the validated, in-bounds candidate and approves before anything touches a record of truth — catching the valid-but-wrong answers a schema and an allowlist cannot.

Constrained output controls the *shape*; the validator confirms the value is *permitted*. Crucially, a grammar guarantees syntax, not meaning — the model can still emit a perfectly valid, perfectly wrong enum value (and the renormalization that enforces the grammar can even nudge quality down). So the model is treated as an untrusted source: its structure is enforced, its choice is bounds-checked, and a human ratifies the substance.

Why RAG grounding is what makes a small model usable

A small model's own "knowledge" is unreliable and unverifiable. **RAG — retrieve-then-read** — flips that: SAS/R first retrieves the short, relevant passage (the study spec, the SOP excerpt, the matching prior finding), hands *only that* to the model, and asks it to answer *from the supplied text*. The model is reduced to reading and labelling a short context it was given, not recalling from training. This both raises accuracy and keeps the context short — the regime small models handle well. Grounded on a short retrieved passage, a small model is useful; asked to answer from memory or over a long document, it is not. RAG reduces, but does not eliminate, hallucination — which is why the validator and the human gate still apply.

Effective vs advertised context window

A model card may advertise a large context window, but a small quantized model's **effective** window — where it still reasons reliably — is much shorter. Accuracy falls off, and information in the middle of a long input is quietly lost, well before the advertised limit. Treat the advertised number as a hard ceiling you stay far below: keep inputs short, retrieve narrowly with RAG, and never rely on the model holding a long document in mind. This is precisely why long summaries and multi-document synthesis sit in the "do not trust" list.

Determinism: pin the decoding AND the environment

For regulated work a run must be reproducible. SAS/R pins the decoding to **greedy sampling (temperature 0) with a fixed seed**, and pins the model identity — **name + quantization + digest** — plus the exact prompt. But temperature 0 and a fixed seed are **not sufficient on their own**: floating-point and batching behaviour can vary across hardware, GPU vs CPU, runtime version, and concurrency, so the same prompt can drift if the environment changes. Reproducibility is therefore a property of the *frozen environment*, not the seed alone: pin the runtime build, the backend (e.g. CPU-only), the context length, and the batch conditions, then **verify by re-execution** and record the result. Treated that way — pinned, constrained, validated, human-gated, and re-run to confirm — the language step is a controlled, qualifiable artifact under a GAMP-5 / 21 CFR Part 11 framework. The point is not that the model is "smart"; it is that the step is documented, bounded, and re-runnable, with the regulated numbers still coming only from the validated engines.

> **The summary.** A small on-device model is safe exactly where the task is short, bounded, structured, and grounded — classification, extraction, routing, similarity, and templated drafting, all behind a schema, a bounds-checking validator, and a human. The moment it must reason across documents, hold long context, write nuanced narrative, or touch a number, it is the wrong tool. SAS/R owns the truth; the small model just helps with the words.

The SLM + SAS/R architecture

This is the same governance as the other companion stacks — a deterministic engine does all the real work, a human approves before anything touches a record of truth, and a Part 11 audit trail records all of it. The only change here is the language helper: instead of cloud Claude behind a gateway, a small open-weight model runs **locally on the box, fully offline**. SAS/R still owns every number; the model only does a narrow language sub-task, and only when called.

Across the 75-component trial-ops stack the assessment assigns each component one verdict for the *language* sub-task: **Ready 12** (the language sub-task sits squarely in the reliable zone), **Guardrails 36** (feasible on-device only with tight scaffolding and a stated residual risk), **Beyond 15** (needs frontier-level reasoning or long-context synthesis a small quantized model cannot do reliably — keep deterministic or escalate to a larger model off-box), and **Platform 12** (pure infrastructure, not a language task). These four sum to 75; they are this assessment's judgement calls, not a measured benchmark. Most useful work falls in the Guardrails tier, which is the honest answer: a small local model earns its place only when it is boxed in.

What an “on-device SLM” actually is

A roughly 1–9B-parameter open-weight model (Qwen3 0.6B–8B, Gemma 4 (E2B/E4B), Phi-4-mini, IBM Granite 4.1 (3B/8B), Llama 3.3/3.2 (8B/3B/1B)), typically 4–8 bit quantized to a `GGUF` file, served by `Ollama` / `llama.cpp` / `LM Studio` on an ordinary workstation or laptop (CPU, or a single consumer GPU). No internet, no telemetry. It is a constrained language sidecar, not the system — and it hallucinates more than a frontier model and is more sensitive to prompt and format, so it never gets handed a number, a long document, or a clinical judgement.

Licences are not all “open source.” “Open-weight” does not mean OSI-approved. Phi-4-mini (MIT), Granite 4.1, Qwen3 and Gemma 4 (Apache 2.0) are permissive, but *Llama 3.x* ships under the *Meta Llama Community License* (custom, with an acceptable-use policy and a large-MAU cap) — which is not OSI open-source. Pin the exact model and read its licence before you qualify it; the choice is a documented, change-controlled decision, not an afterthought.

The loop

Scheduler / SAS·R job (the clock)

Windows Task Scheduler / cron — runs under a service account, survives reboots, logs its own history. Whatever scheduler is already validated in your shop.

↓

Deterministic SAS / R work

data access · ALL computation, checks, counts, reconciliation, stats · data-freshness gate. SAS/R owns every number; nothing numeric ever goes to the model.

↓

SAS·R calls the LOCAL SLM endpoint

PROC HTTP / httr2 → `127.0.0.1` Ollama / llama.cpp. ONE narrow language sub-task only (classify · extract a field · route/triage · draft a templated line), grounded in retrieved text, with a JSON-schema / grammar-constrained output. Set the context length explicitly — the runtime default can be as low as 2–4K tokens and silently truncates over-length input.

↓

SAS·R validator

checks the structured output against the allowed values — valid bucket? required fields present? extracted spans actually present in the source text? On any miss it falls back to the deterministic default and flags for review. The model never decides what is valid.

↓

Human approval queue

a person sees the proposed label / draft / routing beside the evidence and approves, edits, or rejects. Nothing regulated moves without a human.

↓

Write to record + audit log

SAS/R performs every write to the system of record (EDC / CTMS / Smartsheet / eTMF) and records who/what/when/which model + quant + digest — one 21 CFR Part 11 trail.

Division of labor (always)

SAS / R owns	The SLM does (only)
Data access, ALL computation, checks, reconciliation and stats, orchestration and scheduling, the validator, every write, and the audit log.	One short, grounded language sub-task with schema-constrained output, returned for a human to confirm.

Reported and regulated values come only from validated engines — **Pinnacle 21**, **Phoenix WinNonlin**, the **EDC/CTMS** — never the SLM. The model classifies, extracts, routes, and drafts a candidate; it does not compute, reconcile, or report.

Do not trust a small quantized model with: anything numeric or quantitative (math, counts, reconciliation, stats — SAS/R owns ALL numbers, always); multi-document synthesis or long-context reasoning (small models degrade well before their advertised context window, and the local runtime may silently truncate input below it); nuanced clinical / safety / regulatory narrative, judgement, or adjudication; faithful long summaries or cross-correlating many signals. When the *language* work needs real reasoning, a 7B quantized model is not enough — that is the “Beyond” tier: keep it deterministic SAS/R-only, or escalate to a larger local or cloud model (out of scope of on-device SLM).

Reproducibility — a frozen, re-runnable artifact

Pin the model **name + quantization + digest**, fix the decoding (`temperature 0` , `top_k 1` , fixed seed), set an explicit context length, and version the prompt and schema. Record that whole tuple in the audit log. One honest caveat: *temperature 0 plus a fixed seed does not by itself guarantee bit-identical tokens.* On a GPU, output can vary run-to-run because reduction order and batch scheduling are non-deterministic; reproducibility therefore also requires pinning the runtime — ideally CPU inference (or deterministic kernels), a fixed build, a fixed context length, and single-stream serving with no concurrent batching. With the stack pinned that way the step becomes practically reproducible, which is what GAMP-5 / Part 11 asks of a qualified component. Two further points keep it defensible regardless: the SLM's output is never authoritative on its own — the deterministic **SAS/R validator** constrains it to allowed values, and a **human** approves — so the record of truth never depends on the model reproducing a token exactly.

How the “Platform” controls simplify here

An offline local model makes several infrastructure controls cheaper than in the cloud companions:

- **Egress firewall (Presidio / DLP)** — collapses to a physical fact: *the box stays offline, so model egress is impossible.* There is no cloud call to inspect or scrub. (This control becomes “keep the box offline” — itself a verified, change-controlled state, not a free pass.)
- **Model-agnostic gateway** — collapses to *SAS/R calls one local* `127.0.0.1` *endpoint.* No router, no per-model cost ledger.
- **Model-freeze registry** — is just the pinned name+quant+digest (plus runtime build and decoding params) recorded on disk, under change control.

The approval queue, RBAC/secrets, and the audit trail remain as they are in the other companions — SAS/R-owned, human-gated, fully logged.

> **Same governance, smaller model.** If you have read the hybrid and Copilot companions, nothing here is new: deterministic engine does the work, validated tools own every regulated number, a human approves, and everything is logged. The on-device variant just swaps cloud Claude for a small offline model on a narrow set of language sub-tasks — trading capability for sovereignty and a frozen, pinned model, with the residual hallucination risk made explicit per component.

Choosing an on-device model

For this stack the model is the least important decision — the scaffolding around it (constrained output, RAG grounding, the SAS/R validator, the human gate) is what makes a small model usable, and that scaffolding is the same regardless of which open-weight model you pick. So choose conservatively. A handful of well-supported open-weight families all run offline under Ollama or llama.cpp on hardware you already own, and any of them is a reasonable starting point for the narrow language jobs in this stack (classify, extract, route, draft short grounded text). The job here is to pick one that is *licence-clean, well-supported, and small enough to validate* — then prove it on a fixed test set for your specific sub-task. Do not assume a model is adequate because it appears in this table; **capability is established by your validation run, not by the model card.**

Representative on-device open-weight models (June 2026)

Choose for a clean licence first — the scaffolding is identical across models

	On-device sizes	Licence	Review burden
Qwen3	0.6B–8B dense	Apache-2.0 ✓ clean	minimal — OSI-approved
Gemma 4	E2B / E4B (edge)	Apache-2.0 ✓ (new)	minimal — verify it is v4
Phi-4-mini	~3.8B, 128K ctx	MIT ✓ most permissive	minimal
IBM Granite 4.1 enterprise-first default	3B / 8B	Apache-2.0 ✓ clean	minimal — ISO 42001
Llama 3.3 / 3.2	8B / 3B / 1B	⚠ custom community	legal reads full terms

The model is the least important decision: prefer Apache-2.0 or MIT — **Qwen3, Gemma 4, Phi-4-mini, Granite 4.1** are licence-clean; **Llama**'s custom community licence (MAU cap, acceptable-use policy, output-training clause) needs a full legal read — then pick the smallest size that passes your validation set.

Model family	Sizes that run on-device	License & commercial-use note	Good for
Qwen3 (Alibaba)	0.6B, 1.7B, 4B, 8B (dense; also 14B / 32B)	**Apache-2.0 across the entire dense lineup (0.6B–32B)** — clean, OSI-approved. The per-size licence trap that affected Qwen2.5's 3B is gone in Qwen3, but still verify the size you pull. 128K context (32K on the 0.6B/1.7B); dual-mode "thinking" and native tool-calling.	A strong, licence-clean default. 4B/8B for classification, extraction and short grounded drafting; 0.6B/1.7B for cheap routing and triage tagging.
Gemma 4 (Google)	E2B (~2.3B eff.), E4B (~4.5B eff.); also 26B MoE (~4B active) / 31B dense	**Apache-2.0** — new in Gemma 4 (released 02 Apr 2026): Google dropped the custom Gemma Terms of Use, so the family is now OSI-clean. That reverses a long-standing constraint (Gemma 2 and Gemma 3 were custom-licensed), so verify the release you pull is actually Gemma 4.	Explicitly edge-positioned (phones, consumer GPUs). E2B/E4B for on-device grounded drafting and tagging — strong capability, now with a clean licence.
Phi-4-mini / Phi-4-mini-reasoning (Microsoft)	~3.8B (128K context)	MIT — the most permissive licence here, minimal review burden.	Competitive among ~4B models on bounded extraction and short structured tasks; the –

			`reasoning` variant adds multi-step reasoning at the same size. Validate on your task rather than relying on benchmark claims.
IBM Granite 4.1 (IBM)	3B, 8B (also 30B)	Apache-2.0 — clean, OSI-approved, and built for this kind of work. Released 30 Apr 2026; ISO 42001-certified, U.S.-vendor-sourced, up to 512K context, hybrid/dense (no MoE) for predictable latency.	Designed for exactly this stack's jobs: classification, extraction, RAG, JSON/structured output. The enterprise-first, compliance-friendly choice — self-host so sensitive data never leaves the boundary.
Llama 3.3 / 3.2 (Meta)	8B (3.3); 3B, 1B (3.2). Llama 4 also exists but its open sizes are larger MoE, not on-device-small.	Llama Community License — a *custom* licence, not OSI-approved. Commercial use is permitted, but with an Acceptable Use Policy, a "Built with Llama" attribution requirement, the >700M monthly-active-users clause, and a restriction on using Llama or its outputs to train non-Llama models. Legal must read the full terms, not just confirm the MAU threshold.	The most widely-supported family, with the broadest tooling. 8B for extraction/short drafting; 3B/1B for high-volume tagging on phones and light hardware. Accept the custom-licence review.

Sizes, licences, and recommended variants are **as of June 2026 — verify the current version, the exact licence on the specific size you download, and the model card before deploying.** Licences and available sizes change between releases (Gemma moving from a custom licence in Gemma 2/3 to Apache-2.0 in Gemma 4, and Qwen consolidating to uniform Apache-2.0 in Qwen3, are recent live examples), and terms can still differ across sizes within a family.

How to choose — practical guidance

- **Prefer Apache-2.0 or MIT.** Phi-4-mini (MIT), Granite 4.1 (Apache-2.0), Qwen3 (Apache-2.0 across its dense lineup) and Gemma 4 (Apache-2.0 as of the April 2026 release) carry the cleanest, OSI-approved licences and minimise legal review. Llama's Community License is usable for a CRO but is a custom, non-OSI licence with conditions that legal must read in full — do not treat it as "effectively Apache."
- **Pick the smallest model that passes your validation on the task.** Run the candidate against a fixed, representative test set for the specific language sub-task (e.g. "label this finding new-vs-known") and choose the smallest model that clears your acceptance bar. A 3B model that passes is better than an 8B you never validated — smaller is cheaper, faster, and easier to freeze. Do not assume bigger is needed until a smaller one demonstrably fails — but equally, do not assume *any* on-device model passes until it does. If none clears the bar, that is a signal the task is "Beyond" on-device SLM, not a prompt to ship a failing model.
- **Favour instruct / structured-output-capable variants.** Use the `instruct`-tuned build, not the base model, and confirm it follows JSON/grammar-constrained output reliably. In llama.cpp this is enforced with a GBNF grammar (Ollama exposes a JSON/format mode over the same engine); that constrained, schema-shaped output is exactly what the SAS/R validator checks against allowed values before anything reaches the human gate.
- **Quantize deliberately.** Pull a GGUF and choose the quantization for the job: `Q4_K_M` for the smallest footprint and fastest CPU inference (the usual default), or `Q8_0` when you want maximum fidelity and prompt-adherence and can spend the RAM. Heavier quantization shrinks the model but degrades instruction-following and increases hallucination, so re-run your validation set at the quantization you will actually deploy — not at full precision.
- **Pin what you ship, and pin where it runs.** Record model name + size + quantization + file digest, fix decoding (temperature 0, fixed seed, greedy/top_k=1), and freeze the prompt. **Be honest about the limit of this:** temperature 0 plus a fixed seed makes *sampling* deterministic, but it does not by itself guarantee bit-identical

output — GPU floating-point reduction order, batching, and KV-cache layout can still cause small run-to-run variation. For a reproducible, Part 11 / GAMP-5-defensible artifact, also pin the execution environment (e.g. CPU inference, or a deterministic-inference build) and qualify it by re-running the validation set, validating against *behaviour on the test set* rather than asserting bit-for-bit identity. That qualified tuple — not “Llama 8B” in the abstract — is what you register.

Licence and provenance review is an IT/legal gate, not an afterthought. Before any open-weight model is deployed, IT/legal must read the *actual* licence on the specific size *and release* you pull (terms have changed across releases — Gemma 2/3 were custom-licensed while Gemma 4 is Apache-2.0, and Qwen moved to uniform Apache-2.0 in Qwen3; Llama remains a custom, non-OSI licence that differs from Apache/MIT in ways that matter), confirm commercial use is permitted, confirm any pass-through or attribution obligations, and review the model’s provenance and safety documentation. Clear this gate *before* the model reaches a validated environment — treat the model file like any other qualified analytical tool entering the lab, with the licence on record.

Remember what the model is for. None of these models touch a number. They classify, extract, route, and draft short grounded text behind a human gate; SAS/R and the validated engines (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS) own every count, check, reconciliation, and reported value. So choose for licence cleanliness, support, and a footprint you can validate and freeze — not for a leaderboard score. And keep the residual risk in view: small quantized models hallucinate more than frontier models and are more sensitive to prompt and format, which is exactly why the constrained output, the SAS/R validator, and the human gate are non-negotiable. When the language work genuinely needs real reasoning or long-context synthesis, no choice from this table fixes it: that work stays deterministic SAS/R-only or escalates to a larger local or cloud model, which is out of scope here.

Project & Timeline Management

Status drafting, action capture, vendor oversight, timeline notes — mostly short classification/extraction/drafting a small model can do behind a human gate. SAS/R owns all the date math and the writes.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
Milestone & critical-path tracking agent	⚠ Guardrails	**SLM:** Per flagged task, emit a one-bucket severity/at-risk label (from a fixed allowed set) and a short templated 'task X moved onto critical path, float now N days' blurb filled ONLY from SAS/R-supplied per-delta values — no free-form 'why it matters' essay, no cross-task prioritization. **SAS/R:** Pull the Smartsheet plan via API, run all critical-path/float/dependency recompute and the baseline diff, decide which tasks crossed thresholds, RANK the flagged tasks by float deterministically (prioritization is math, not SLM reasoning), validate every bucket label against the allowed set, write nothing without PM approval, and own the audit log. Binding constraint: 'why it matters / prioritize' is multi-signal reasoning over a changing dependency graph — a 7B quant model does this only shallowly and will invent rationale, so prioritization must be SAS/R float-ranking and the SLM is confined to per-delta templated blurbs + fixed-bucket labels behind the PM gate.
Study-event timeline auto-update & notification (data cut, DB lock, milestone slip)	⚠ Guardrails	**SLM:** Compose a short, templated notification ('you are now blocked on X; date moved to Y') by slotting SAS/R-supplied blocked-party names and recomputed dates into a fixed template, and tag each alert with the correct role/channel routing bucket — the model copies dates, never derives them. **SAS/R:** Run the Smartsheet dependency recompute, determine who is downstream/blocked and the new dates, hold all contractual/regulated-milestone commits for lead approval, validate routing labels against the allowed set, and own every write and the audit trail. Binding constraint is stakes, not language difficulty: the templated alert + role routing sits in the reliable zone, but regulated/contractual milestone consequences mean the recompute and the blocked-party list must be SAS/R and dates stay proposed-only behind the lead's gate.
Status-report generation (study & program)	✕ Beyond	**SLM:** None at on-device scale beyond optional single-metric per-slot blurbs; a small quantized model cannot weave milestones + enrollment + risks + aging + vendor status into a faithful program-level narrative without hallucinating cross-section claims. **SAS/R:** Pull every metric from systems of record, fill the house-template slots deterministically, flag any RED metric for human verification against the source, assemble the report skeleton, and own all numbers — none are ever produced or transformed by the SLM. Binding constraint: faithful multi-document synthesis into a long report is exactly where small quantized models degrade and fabricate — keep template-fill deterministic SAS/R and escalate any cross-section narrative to a larger local/cloud model, not an on-device SLM.
Action-item & decision extraction from email/meetings	✅ Ready	**SLM:** From a SHORT chunked passage, extract action items (owner, due-date, text) and logged decisions as JSON-schema-constrained fields, classify each into the fixed action/decision taxonomy, and flag near-duplicates of existing log rows. **SAS/R:** Chunk long transcripts/threads so the model never sees long context, run the dedupe/similarity match against the action log, validate every extracted owner/date against allowed values, and write only owner/PM-confirmed rows with full audit. Binding constraint is transcript length: field/NER extraction + bucket classification +

		near-dup grouping over short passages is the canonical SLM-reliable zone, so the only requirement is that SAS/R chunk aggressively to keep the model out of long context — a clean on-device win behind the confirm gate.
Meeting agenda & minutes drafting	⚠ **Guardrails**	**SLM:** Assemble the agenda by templating SAS/R-retrieved open actions/risks/carryover; for minutes, emit only short per-SEGMENT decision/action extracts from one chunked transcript slice — never a free-form whole-meeting summary. **SAS/R:** Pull open items/risks/carryover from Smartsheet/RAG, segment the transcript into short slices, cross-link extracted decisions to action rows deterministically, and hold all artifacts as drafts until the chair approves. Binding constraint: this component bundles agenda templating (reliable) with faithful full-transcript minutes (a 'faithful long summary' that is genuinely Beyond on-device) — feasible only by amputating minutes down to short per-segment extracts; the free-form-minutes ambition must be escalated, and the chair must review before circulation.
Vendor/CRO oversight & deliverable tracking	⚠ **Guardrails**	**SLM:** Classify each deliverable into an upcoming/overdue/at-risk bucket given SAS/R-computed dates, and draft a short templated oversight-log entry or vendor follow-up email grounded in the retrieved SOW/SLA clause text. **SAS/R:** Track deliverable/SLA dates against SOW obligations, compute overdue/at-risk status and all date math deterministically, retrieve the governing contract clause for grounding, validate status labels, and keep escalation/acceptance as human decisions. Binding constraint: status bucketing and RAG-grounded chasers are reliable, but SLA-clause INTERPRETATION carries contractual nuance a small model can misread — pin all dates to SAS/R, force every draft to quote the retrieved clause, and gate escalation/acceptance to the oversight lead.
Resourcing & capacity conflict detection	⚠ **Guardrails**	**SLM:** Given SAS/R-detected over-allocation/contention windows, draft a short candidate leveling suggestion (re-sequence, shift, add) as one human-reviewed option — proposing, never optimizing or reassigning anyone. **SAS/R:** Read allocations across studies, recompute timelines, deterministically detect over-allocation and contention windows (all date/effort math), and require resource-manager approval before any re-level becomes the plan. Binding constraint: conflict detection is pure date/effort math owned entirely by SAS/R; the leveling-recommendation narrative is light reasoning a small model can draft but cannot be trusted to optimize, so it must be presented as one of several human-chosen options, not 'the' answer.
Risk register upkeep & RBQM linkage	⚠ **Guardrails**	**SLM:** Draft a candidate risk-register entry from an issue/slip or a SAS/R-passed KRI/QTL breach signal, classify status/category into the fixed taxonomy, and PROPOSE (not decide) a likelihood/impact bucket for human confirmation. **SAS/R:** Receive RBQM KRI/QTL breach signals, compute all thresholds and scoring math, assemble the review packet, validate taxonomy/scoring labels against allowed values, and route QTL-breach actions through the quality decision path with sign-off held to humans. Binding constraint: drafting and classifying a candidate entry is reliable, but likelihood/impact scoring and mitigation adequacy are nuanced quality/safety judgement under ICH E6(R3) — the SLM may only propose a bucket; scoring math stays SAS/R and acceptance stays with the quality/PM lead.
Budget & PO / accrual tracking	⚠ **Guardrails**	**SLM:** Draft short budget-vs-actual commentary and flag-narrative ONLY from variances already computed by SAS/R/ERP, grounded in the retrieved contract/payment-schedule clause — the model touches and restates no numbers it did not receive verbatim. **SAS/R:** Reconcile invoices/POs against budget lines and milestone-payment schedules, compute every accrual and variance, source all reported actuals

		from the finance/ERP system of record, validate that the draft's figures match the supplied values, and keep the SLM entirely out of the commit path. Binding constraint: RED tier and all-numeric — reconciliation and reported actuals are SAS/R/ERP-only by rule, so the SLM is confined to narrating pre-computed variances behind a finance approval gate; its sole residual risk is the prose, and the validator must reject any figure it altered.
Cross-study portfolio rollup	❌ **Beyond**	**SLM:** None at on-device scale beyond a single-study one-liner; a small quantized model cannot cross-correlate milestones, slip, risk heat, enrollment, and budget across many studies into a coherent portfolio narrative and exception list. **SAS/R:** Aggregate cross-sheet metrics from every study's system of record, compute slip/risk/budget health, rank and select the exceptions deterministically, and build the dashboard table; the cross-study narrative is what exceeds the SLM. Binding constraint: many-document, long-context cross-correlation is the precise degrade zone for small quantized models — keep the rollup and exception ranking deterministic SAS/R and escalate any portfolio narrative to a larger local/cloud model.
Action-item SLA chasing & escalation	✅ **Ready**	**SLM:** Draft a short, polite reminder email per SAS/R-identified due/overdue action and a brief escalation summary line for SLA-breaching items, routing each to the right owner/queue bucket. **SAS/R:** Scan the action/decision log, compute due/overdue/SLA-breach status with all date math, validate routing targets against allowed values, send only per policy, and reserve the escalate-or-not decision for the PM. Binding constraint is minimal: short templated drafting plus owner/queue routing over SAS/R-computed due-dates is squarely reliable, and the only nuance — whether to escalate — is explicitly held human, leaving low residual risk; a clean on-device win.
Milestone-slip root-cause & schedule-recovery proposal	❌ **Beyond**	**SLM:** None reliably on-device: correlating likely drivers across linked actions, risks, vendor deliverables, and status notes is multi-signal diagnostic reasoning a 7B quant model cannot do faithfully. **SAS/R:** Confirm the slip, gather the linked actions/risks/deliverables/notes, run all recovery impact modeling in Smartsheet (fast-track/re-sequence/resource-add timelines), and hold scenario selection for the PM/lead. Binding constraint: root-cause correlation across many heterogeneous signals plus scenario reasoning is genuine frontier-level work — keep the impact math in Smartsheet and escalate the diagnosis/recovery narrative to a larger local or cloud model, not an on-device SLM.

eTMF & Document Control

Document classification to the TMF model, metadata/expiry extraction, SOP Q&A over RAG — a strong fit for a small model on short passages with constrained output. Filing of record stays human + validated eTMF.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
Auto-classify & file to the DIA TMF Reference Model	⚠ Guardrails	**SLM:** After SAS/R-invoked OCR, classify the document into ONE allowed TMF Reference Model v3.2.0 Zone>Section>artifact ID via grammar-constrained output, choosing only from a SAS/R-supplied SHORTLIST of candidate IDs (retrieval-narrowed from the full taxonomy), and extract short metadata fields (study, site, country, document date, version) as schema-validated NER, each with its own confidence score for human confirmation **SAS/R:** Runs/queues OCR, owns the authoritative TMF taxonomy enumeration, retrieval-narrows the hundreds of artifact IDs down to a small candidate shortlist BEFORE the SLM sees them, validates the SLM's proposed artifact ID and every metadata field against allowed values/known site-country lists, computes confidence routing thresholds, writes nothing to the eTMF, and logs the draft filing for the human gate Binding constraint is the very large label space (hundreds of artifact IDs) read from whole-document OCR context — that is NOT short-passage bucket classification, so constrained output only guarantees a VALID id, never a CORRECT one; a SAS/R retrieval-narrowed candidate shortlist plus the allowed-value validator and the RED filing gate are mandatory, and mis-classification remains the residual risk.
TMF completeness / QC & missing-document detection	⚠ Guardrails	**SLM:** Only the templated language wrapper: draft a short, grounded gap-list narrative and QC summary sentence from the missing/overdue artifacts SAS/R already computed (no counting, no percentages of its own) **SAS/R:** Owns ALL of it numerically — joins filed artifacts to the expected-document index per phase/site, computes per-zone completeness %, enumerates missing/overdue/placeholder items, detects wrong-version/unsigned/wrong-date defects deterministically, and never lets the SLM touch a status field This is fundamentally reconciliation and counting, which is SAS/R-only territory; the SLM may only phrase the result, and even then a hallucinated count must be impossible — every number in the narrative is templated from SAS/R-supplied values, so the binding residual is only narrative drift, contained by the bind-the-numbers design and the human gate.
Expiring-document alerts (CV / training / license / certs)	✅ Ready	**SLM:** Extract the expiry date (and document type/holder) from a SHORT credential document (CV, GCP cert, license, lab cert, IATA cert) as a schema-validated date field, and draft the templated reminder notice text for the responsible role **SAS/R:** Owns the date arithmetic and 90/60/30-day threshold logic, maintains the credential-currency register, validates extracted dates (format/plausibility) before they drive any alert, schedules/sends alerts, and keeps renewals manual with the audit log Squarely in the reliable zone — short-passage date NER plus templated drafting — with all time math owned by SAS/R; the only residual is a mis-read date, caught by SAS/R plausibility checks and the alert-only (no auto-renewal) design, so the binding constraint is extraction accuracy on a short passage, which is the SLM's strength.

SOP / WI / study-document ingestion into the RAG knowledge base	✅ Ready	**SLM:** None generative — at most produce local embedding vectors for chunks; it authors nothing and makes no free-text decisions **SAS/R:** Owns parsing, chunking, the metadata stamp (doc ID, version, effective date, owner, applicable studies), writing to the permissioned vector store, and the supersession logic that retires the prior version's chunks driven by the DMS state of truth This is a deterministic indexing pipeline plus embeddings with essentially no generative hallucination surface; the binding constraint is stale-version retrieval, handled by SAS/R tying ingestion/retirement to the validated DMS 'effective' state rather than any model judgement — a clean on-device win.
Compliance Q&A over SOPs/WIs ("what does SOP-12 require for X?")	⚠️ Guardrails	**SLM:** Read the SAS/R-retrieved current-effective SOP/WI chunks and answer the requirement in short grounded extractive text, quoting and citing the exact doc ID, version, and section, and flagging when multiple/conflicting or superseded documents appear in the retrieved set **SAS/R:** Owns retrieval and re-ranking from the permissioned vector store, supplies only current-effective chunks, runs a citation validator that verifies every cited doc ID/version/section actually exists in the retrieved set (rejecting any answer that cites outside it), and routes regulatory-binding interpretation to QA/RA Grounded extractive Q&A is feasible on-device, but conflicting-document detection and any interpretive edge are where a small model over-claims, so the citation validator, the explicit 'verify against the controlled document' framing, and the QA/RA escalation are the load-bearing guardrails; the binding constraint is over-claim beyond the retrieved text, blocked by the validator.
Version / superseded-document control & reference reconciliation	⚠️ Guardrails	**SLM:** Write a short grounded plain-language summary of what changed between old and new versions (strictly from the SAS/R-computed diff) and draft the reconciliation checklist text; propose (not execute) the 'superseded' label **SAS/R:** Computes the deterministic text diff, finds every downstream document and TMF reference that cites the prior version by exact match/search, determines which trainings re-trigger by rule, and executes the actual supersession status write only after the human approves (RED on the status write) The diff and the cite-finding are deterministic and must stay in SAS/R; the SLM only narrates the SAS/R-supplied diff, and because a mischaracterized change has compliance impact, the binding constraint is change-summary fidelity — contained by grounding the summary on the computed diff and gating the status write to the human/validated system.
Inspection-readiness gap analysis	❌ Beyond	**SLM:** None for the judgement core; at most draft individual templated finding lines from SAS/R-computed signals — but the prioritized, cross-correlated readiness narrative must NOT come from a small quantized model **SAS/R:** Computes every input signal (zone completeness, overdue/placeholder, unsigned/out-of-window, expiring credentials, superseded references, inspector hot-spot rules) and assembles the prioritized remediation plan with owners deterministically This cross-correlates many heterogeneous signals into a nuanced, high-stakes readiness judgement and executive narrative — long-context multi-signal synthesis small models do unreliably; the binding constraint is exactly that synthesis, so keep the scan deterministic SAS/R-only and ESCALATE any generated narrative to a larger local or cloud model on de-identified data.
Redaction / PII-PHI detection before any cloud use	◇ Platform	**SLM:** None as the gate — Presidio (deterministic, incl. MedicalNER and DICOM/image redaction) is the enforcement point; the on-device SLM only optionally adds context-aware secondary flags and is never the sole arbiter of

		'safe' **SAS/R:** Owns the gateway enforcement: runs Presidio, blocks the cloud route and forces local processing when PII/PHI is found, produces the de-identified variant with a logged mapping, and records the routing decision in the audit log This is egress-control infrastructure, not an SLM language task, so it is Platform; in the on-device-SLM+SAS/R variant the control simplifies dramatically — if the box stays fully offline there is no cloud route to gate at all and the control collapses to 'the box stays offline,' with Presidio remaining the technical guarantee plus a human/QA sample check on de-identification adequacy.
Document QC linter (signatures, dates, legibility, completeness fields)	⚠ Guardrails	**SLM:** Extract short presence/identity fields needed for the checks (is a signature/date present, which form version, are required fields populated) as schema-validated outputs, and draft the templated QC finding note routed back to the originator **SAS/R:** Owns the signature-vs-event date-window arithmetic, page-completeness counts, legibility/scan-quality metrics (deterministic image checks), the correct-version lookup, and setting the official accept/reject QC status only via the validated eTMF after human confirmation All numeric/temporal checks (date windows, page counts, scan-quality) are SAS/R; the SLM only does presence-extraction and note drafting, so the binding constraint is mis-extraction of a field, contained by constrained output plus the document-controller accept/reject gate — it explains, it never passes/fails the record.
Email & attachment auto-routing to the eTMF intake	⚠ Guardrails	**SLM:** Classify an incoming email/attachment into a fixed artifact-type bucket (signed form, site cert, vendor-qualification doc, etc.) and extract short study/site context for routing, all grammar-constrained against a SAS/R-supplied bucket list **SAS/R:** Owns mailbox polling, truncates/segments long or multi-topic threads before the SLM sees them, validates the proposed artifact type and study/site against allowed values, routes the attachment into the downstream classify/QC pipeline with proposed metadata, and keeps actual filing as a separate human-approved step Downgraded from Ready: real email bodies are variable-length, noisy, multi-topic threads with mixed attachments, so this is NOT clean short-passage classification and inbox triage is a classic spot where context length and ambiguity degrade a small model; the binding constraint is mis-classification on messy input, mitigated by SAS/R thread-truncation, allowed-value validation, and the intake-queue reviewer — routing is reversible and filing is gated downstream, so stakes stay low.
Translation / certified-copy & localization control	⚠ Guardrails	**SLM:** Detect the document language and type (short schema-validated labels), and draft a ROUGH working translation/summary for reviewer context only (explicitly not of record); it never produces the certified translation **SAS/R:** Owns the deterministic pairing/cross-link check (is a certified translation present and correctly linked to its source/certified-copy artifact), flags missing or unlinked pairs, and keeps the certified translation of record as a validated human/vendor deliverable in the system of truth The deterministic pairing check is the safe load-bearing value; on-device small-model translation is only gist-quality and genuinely unreliable for regulatory text, so the binding constraint is translation fidelity — it stays Guardrails ONLY because the SLM output is firewalled to working/never-of-record and the certified translation remains a vendor deliverable; without that never-of-record framing the translation task itself would be Beyond.
TMF audit-trail & filing-activity summarizer	✗ Beyond	**SLM:** At most phrase a few SAS/R-computed bullet metrics into a sentence; the cross-correlated TMF-health narrative over many audit-log rows must NOT come from a small quantized model

		SAS/R: Reads the immutable Part-11 audit trail, computes ALL metrics (filing volumes, backlog age, who-filed-what, recurring rejection reasons, bottleneck trends) deterministically, and never edits the trail; reported numbers tie back to the system of record This is faithful long-summary and multi-signal synthesis over many log records — exactly where small models lose fidelity and invent trends; the binding constraint is faithful long-context aggregation, so keep aggregation deterministic SAS/R-only and ESCALATE the executive narrative to a larger local or cloud model on de-identified data, with the TMF lead reviewing before governance.

Data-Validity & Recurrent QC

SAS/R (and Pinnacle 21 / Phoenix) produce the authoritative findings and every number. The small model only *labels, clusters new-vs-known, and drafts a query text* over the finding — schema-constrained and SAS/R-validated. It never touches a value.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
Scheduled CDISC conformance run + AI finding-triage (SDTM/ADaM/Define-XML)	⚠ Guardrails	**SLM:** Per individual Pinnacle 21 finding (one at a time): emit a rule/domain bucket label, echo the SAS/R-supplied NEW-vs-repeat tag, and draft a short RAG-grounded plain-language explanation plus a candidate disposition label (defect vs explained-in-cSDRG/ADRG) from a closed set — for human confirmation only. **SAS/R:** Runs the validated Pinnacle 21 report (record-of-truth), computes the diff against the last clean run, does the cross-finding clustering/dedup itself, hands the SLM one finding at a time, validates every cluster/disposition tag against an allowed enum, routes by owner, and writes the audited disposition log. Binding constraint is STAKES plus latent synthesis: per-finding labeling and templated drafts are reliable, but 'real defect vs expected' is reviewer-guide nuance and the disposition feeds a regulatory cSDRG/ADRG, so keep clustering in SAS/R, cap the SLM to one finding, and gate every disposition.
Cross-form / edit-check / logical-consistency surveillance	⚠ Guardrails	**SLM:** On ONE pre-flagged participant row at a time, classify error-vs-explainable-protocol-event into a fixed bucket, extract the named fields, and draft a one-paragraph candidate query for human approval. **SAS/R:** Runs the validated edit-check / cross-form SQL battery that deterministically produces every flag, hands the SLM only the single flagged row, validates bucket/route against allowed values, and issues nothing until DM approves. Binding constraint is NUANCE: error-vs-explainable-protocol-event is clinical-judgement-adjacent, so it stays Guardrails — one flag at a time, RAG-grounded in the protocol, every query human-gated — and never promotes to unattended.
PK timing (actual vs nominal) and BLQ/LLOQ rule checking	⚠ Guardrails	**SLM:** Label each SAS/R-computed timing-deviation/BLQ flag as expected/ambiguous/needs-PK-review against the sampling schedule via RAG, and draft the discrepancy note for the PK scientist — no numeric output. **SAS/R:** Computes all actual-minus-nominal deviations and applies SAP/PK-plan BLQ/LLOQ handling deterministically; Phoenix WinNonlin re-derives any analysis value; SAS/R validates the SLM's labels against an enum. Binding constraint is STAKES via over-confident PK reasoning: the numbers and BLQ logic are firmly SAS/R, so restrict the SLM to flag-labeling plus note drafting behind a PK-scientist gate and keep it out of every analysis value.
Lab / IXRS(IRT) / PK-sample reconciliation	⚠ Guardrails	**SLM:** For one unreconciled record pair, classify the mismatch into a fixed set (missing sample, unit mismatch, duplicate, late transfer, mis-keyed accession), propose a likely root cause, and draft the reconciliation finding plus route. **SAS/R:** Performs 100% record-count/key/value matching across central-lab, IXRS, and PK manifests (the matched dataset is the validated record), supplies only the unreconciled set one item at a time, validates mismatch-class and route against allowed values, and logs resolutions after human confirmation. Binding constraint is root-cause CONFABULATION feeding PK sample accountability: mismatch-type classification over a short record pair is reliable, but a plausible-wrong

		root-cause guess is a real failure mode, so it stays Guardrails with the closed class set plus mandatory DM/PK confirmation — not Ready.
Query-management oversight, aging & SLA escalation	❌ **Beyond**	**SLM:** none for the analytics — the SLM does NOT compute aging buckets, SLA breaches, lock-threat predictions, or summarize the multi-query landscape; if used at all it only verbalizes one already-ranked item into an escalation email behind a human gate. **SAS/R:** Pulls open-query metadata and deterministically computes aging buckets, SLA-breach flags, milestone-risk ranking, and the prioritized worklist; the SLM is given finished outputs only. Binding constraint is FORBIDDEN MATH plus multi-query synthesis: as written the component hands the SLM aging/SLA/lock-threat computation and landscape summarization, so keep ALL analytics DETERMINISTIC SAS/R-only (escalate any cross-query narrative to a larger model) and demote the SLM to at most a single per-item email draft.
Duplicate / outlier / range / digit-preference checks	⚠️ **Guardrails**	**SLM:** For one statistically-flagged value at a time, classify plausible-extreme-physiology vs likely-transcription-error into a fixed bucket against the variable's expected range and the participant's own prior values (RAG), and draft a query or 'no-action, explained' note. **SAS/R:** Computes all range violations, robust-z/IQR outliers, duplicate detection, and digit-preference statistics deterministically, feeds the SLM one flag plus the participant's trajectory, validates the bucket, and feeds digit-preference results to central monitoring. Binding constraint is STAKES: all quantitative work is SAS/R, but calling an extreme-but-real physiological value a transcription error is a genuine failure mode, so it stays Guardrails — short retrieved trajectory, one flag, human confirms each disposition.
Data-quality KRI / QTL trend dashboard upkeep (RBQM)	❌ **Beyond**	**SLM:** At most draft a single per-KRI caption from one SAS/R-computed metric row; it does NOT author the cross-KRI RBQM narrative or judge signal-vs-noise across the panel. **SAS/R:** Computes all KRI/QTL values, breach status against ICH E6(R3) acceptable ranges, and which metric moved; owns the validated metric layer driving every threshold. Binding constraint is LONG-CONTEXT MULTI-KRI SYNTHESIS with signal-vs-noise judgement — exactly what a quantized 7-9B model degrades on — so ESCALATE the decision-grade narrative to a larger local or cloud model; on-device, the SLM is limited to single-metric captions.
Statistical / central-monitoring anomaly signals (site outliers, enrollment & visit anomalies)	❌ **Beyond**	**SLM:** At most draft a single-signal follow-up memo from one SAS/R-ranked anomaly; it does NOT correlate signals across KRIs or adjudicate data-integrity-vs-operational cause across the site panel. **SAS/R:** Runs the validated CSM engine computing site-vs-site outlier statistics, variance/mean anomalies, enrollment/visit-pattern and over-uniformity signals, and the ranked signal list; supplies de-identified summaries only. Binding constraint is HIGH-STAKES MULTI-SIGNAL CORRELATION on fraud-adjacent signals — cross-KRI integrity-vs-operations adjudication is beyond a small quantized model — so ESCALATE that synthesis to a larger model and restrict the on-device SLM to one memo per already-ranked signal behind a central-monitor gate.
Pre-lock data-review listing generation, diff & sign-off packet	❌ **Beyond**	**SLM:** At most draft an individual templated packet line (e.g., one open item mapped to its query) from a SAS/R-supplied diff; it does NOT author the full what-changed summary or judge SAP-completeness. **SAS/R:** Generates the validated blinded listings, computes the diff against the prior review run, maps open items to queries/dispositions, and deterministically checks which SAP-required checks are evidenced. Binding constraint is LONG-CONTEXT SYNTHESIS AT LOCK STAKES: faithful multi-

		listing diff summarization and lock-readiness/SAP-gap judgement are beyond a small model, so keep the diff and completeness checks DETERMINISTIC SAS/R-only, ESCALATE any prose summary to a larger model, and the biostatistician owns sign-off.
Discrepancy triage, classification & routing to DM	⚠ Guardrails	**SLM:** Per single discrepancy, classify type/severity/likely-owner into fixed buckets, tag near-duplicates for SAS/R to merge, draft the RAG-grounded query text, and propose a route from a closed set. **SAS/R:** Aggregates flags from all validated checks, performs the actual cross-flag de-duplication/merge on the SLM's similarity tags, validates class/severity/route against allowed values, assembles and ranks the single DM worklist, and keeps resolution/closure in the validated system. Binding constraint is BREADTH of the unifying layer: short-text classification, similarity tagging, and templated drafting are reliable, but this touches many sources, so cap context to one discrepancy, let SAS/R own the cross-flag dedup and ranking, and gate routing — promising but not unattended.
MedDRA / WHODrug auto-coding QC & uncoded-term triage	⚠ Guardrails	**SLM:** From one short verbatim string, flag a suspicious auto-code, surface the uncoded/queried term, and suggest candidate MedDRA/WHODrug code(s) WITH the dictionary path as a human-confirmation hint, and draft the coding query. **SAS/R:** Runs the validated versioned dictionary/coding tool holding every code-of-record, supplies the QC output and verbatim terms, validates that any SLM-suggested code is a real path in the loaded dictionary, and never lets the SLM write the assigned code. Binding constraint is SAFETY-RELEVANT CONFABULATION: small models invent plausible-wrong codes, so the dictionary-path validation plus mandatory human-coder confirmation are load-bearing, keeping this Guardrails rather than Ready.
SDTM<->Phoenix<->ADaM PK numeric cross-traceability reconciliation	⚠ Guardrails	**SLM:** On an already-detected SAS/R mismatch, classify WHERE the chain likely broke into a fixed set (mapping, BLQ rule, rounding, parameter-code mapping, RELREC/time-after-dose), explain briefly, and draft the finding to the PK scientist/programmer — never a number. **SAS/R:** Runs validated reconciliation confirming PK parameters match across PP <-> Phoenix (engine of record) <-> ADPP and that RELREC PC<->PP and time-after-dose are consistent; all PK numbers re-derive from Phoenix and the SLM never computes or alters a value. Binding constraint is the HIGHEST STAKES (CSR-bound PK), but the SLM's task is purely qualitative break-point localization on an already-flagged mismatch, so it clears Guardrails only with the closed cause-set, zero numeric authority, and a mandatory PK-scientist plus lead-biostat gate.

Trial Monitoring & RBQM

KRI/QTL signal labelling, deviation classification, MVR action extraction. Nuanced safety narrative and multi-signal synthesis are *Beyond* a small model — SAS/R computes, the model tags, humans decide.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
KRI/QTL definition & specification drafting	❌ Beyond	**SLM:** none — the metric-spec drafting genuinely requires synthesizing protocol + risk assessment + SOP library into defensible definitions, numerator/denominator logic, rationale, and candidate thresholds; escalate to a larger local model (or cloud only on de-identified protocol text) **SAS/R:** RAG retrieval over the protocol/SOP corpus; the validated CSM platform computes every metric and every threshold of record; SAS/R logs provenance and routes any escalated draft to the risk committee and biostat lead. No number, threshold, or definition of record originates in any model. Binding constraint is multi-document synthesis plus regulated CtQ judgement across protocol+risk-assessment+SOPs — long-context reasoning that a 7B quantized model degrades on well before its context window, with high adjudication stakes (a wrong KRI definition mis-aims the whole monitoring plan). Keep only deterministic/templated catalog scaffolding local; the real drafting is Beyond on-device.
Threshold-breach signal triage & narrative	⚠ Guardrails	**SLM:** Tag breach severity/priority into a fixed allowed-value bucket set and draft a short templated breach note (what breached, direction of trend) strictly grounded in the validated platform's already-computed flagged record **SAS/R:** The validated CSM/RBQM platform detects the breach and computes magnitude, trend, and every number; SAS/R validates the SLM's severity tag against the allowed enum, routes to the queue, and writes nothing to the breach-of-record without the human gate. No magnitude or count is ever generated by the model. Binding constraint is that the breach numbers and trend must be pre-computed by the validated tool — feasible only with constrained output and the record supplied; residual risk is the SLM inventing candidate causes or restating magnitude, so any cause text stays clearly a draft and numbers never originate in the model.
Central statistical monitoring signal interpretation (site outliers, digit preference, anomalies)	❌ Beyond	**SLM:** none for the synthesis — cross-correlating outlier scores, digit-preference, variance/correlation, date-implausibility and AE-under-reporting into one fraud/quality story needs frontier reasoning; if any SLM use at all, restrict to one-finding-at-a-time templated captions under separate Guardrails, never the cross-signal narrative **SAS/R:** The validated/frozen stats engine computes all outlier statistics, digit-preference tests, variance/correlation anomalies and date-implausibility flags; SAS/R clusters findings deterministically by site and supplies only the computed ranking. The model never computes or confirms a statistic. Binding constraint is multi-signal correlation into a coherent fraud/integrity narrative with high adjudication stakes — exactly the cross-correlating-many-signals failure mode small quantized models hit. Keep all statistics SAS/R-only and escalate the integrative narrative to a larger model.
Protocol-deviation surveillance & classification	⚠ Guardrails	**SLM:** Classify a single short deviation passage into major/minor and one fixed category (eligibility, IP, visit-window, consent, procedure), flag likely-important per ICH E6(R3), and draft the log-entry text **SAS/R:** RAG to the protocol section map; SAS/R validates the chosen class/category against the allowed enum, attaches the cited protocol section, and writes only through the validated deviation-log system after QA confirmation. No status of record is set by the model. This is short-passage classification into a fixed bucket set — the SLM's reliable zone

		— but the 'major/important' determination carries direct regulatory weight, so the deterministic enum validator plus the human QA gate are non-negotiable; without them this would slide toward an over-trusted high-stakes call, which is why it is Guardrails not Ready.
Safety signal triage (AE/SAE volume, lab shifts)	❌ **Beyond**	**SLM:** none — clinical-safety nuance and causality-adjacent prioritization on unblinded data is explicitly outside small-model competence; either leave triage to deterministic SAS/R ranking + medical review or escalate the note to a larger LOCAL model (never cloud, given unblinded data) **SAS/R:** The validated safety DB/EDC computes all AE/SAE volumes and lab-shift numbers; SAS/R assembles the flagged inputs, enforces that no number originates in any model, and routes to the safety physician. The model is kept entirely off the causality and number paths. Binding constraint is the highest-stakes narrative class in this area — clinical-safety judgement on unblinded data where a small model's hallucination or mis-prioritization is directly patient-relevant. Keep the SLM off it; this is Beyond on-device.
Site performance & enrollment-metric reporting	⚠️ **Guardrails**	**SLM:** Draft one-site-at-a-time scorecard commentary and recommended-action phrasing grounded only in the validated per-site metrics (enrollment vs plan, screen-fail, query aging, deviation rate, entry lag) **SAS/R:** CTMS/EDC compute all metrics; SAS/R passes de-identified per-site aggregates, validates that the narrative cites only supplied figures, and performs every cross-site/portfolio rollup deterministically before clinical-ops review. The model never aggregates across sites. Binding constraint is that the source scope includes a multi-site/portfolio rollup — exactly the multi-row synthesis where small models drop or transpose figures. Per-site templated summarization is in reach, so constrain the SLM to single-site drafting and force all cross-site aggregation into SAS/R; that constraint is what keeps it Guardrails rather than Beyond.
Enrollment & visit-anomaly detection narrative	⚠️ **Guardrails**	**SLM:** Narrate a single already-detected anomaly (enrollment spike/cluster, visit-window violation, implausible timing), suggest candidate causes, and draft a one-line monitoring follow-up **SAS/R:** Validated analytics perform all detection math and timing checks; SAS/R supplies the one discrete flagged event, validates the routing tag, and assigns follow-up only through the human gate. The model never computes or confirms the anomaly. Binding constraint is candidate-cause speculation: detection is fully deterministic so the SLM only explains one pre-found event (within reach), but the speculative cause is the residual hallucination risk, so it stays a clearly-labeled draft and the model must never imply it computed or confirmed the anomaly.
CAPA drafting & tracking	⚠️ **Guardrails**	**SLM:** Draft a templated CAPA record (problem statement, candidate root-cause options, proposed corrective/preventive actions, owner/due-date placeholders) and short aging-status note text from a single finding **SAS/R:** RAG to the CAPA SOP; SAS/R computes all overdue/aging dates, tracks open CAPAs, validates owner/date fields, and writes closure only in the validated quality system after QA approval. All date arithmetic is deterministic, never the model's. Binding constraint is that root-cause options are suggestions only — single-finding templated drafting grounded in the SOP is in the reliable zone, but effectiveness checks, closure judgement, and every date computation stay deterministic and human so the model never implies a verified root cause or a real due date.
DSMB / safety-review packet assembly	❌ **Beyond**	**SLM:** none on-device — faithful long-form synthesis of the unblinded packet cover narrative and prior-action follow-up summary needs a larger model; escalate to a larger LOCAL model only, never cloud given the unblinded data

		SAS/R: Validated tools produce every TLF; SAS/R pulls and orders the validated tables/listings/figures, assembles the packet shell deterministically, and keeps all numbers out of any model's hands. The packet structure is templated by SAS/R, not narrated into existence by the SLM. Binding constraint is faithful long-form synthesis over an unblinded safety packet — the long-summary + high-stakes territory where small models drop or fabricate findings. Keep collation and TLF assembly deterministic in SAS/R; the cover narrative is Beyond on-device and must stay on-prem if escalated.
Monitoring-visit report (MVR) summarization	❌ Beyond	**SLM:** single-MVR structured extraction (open issues, action items, deviations noted, site responsiveness) could run under tight Guardrails, but the cross-MVR recurring-theme rollup as scoped needs a larger model — escalate the rollup **SAS/R:** SAS/R supplies de-identified MVR text, enforces the output schema on any extraction, and stores drafts pending CRA/clinical-lead confirmation of findings of record. SAS/R, not the model, does any cross-report aggregation of themes. Binding constraint is faithful long-document summarization plus cross-MVR theme synthesis — small quantized models drop or fabricate findings well before their advertised context limit, and the portfolio rollup is multi-document synthesis. As scoped (full MVR + cross-site rollup) this is Beyond; only a narrow single-MVR schema extraction would be Guardrails.
MVR follow-up & action-item tracking	⚠️ Guardrails	**SLM:** Extract discrete action items, owners, and due dates from a short MVR passage and draft short reminder text for aging items **SAS/R:** SAS/R computes all aging/overdue dates, opens/updates only the non-regulated tracker via MCP, validates owner/date fields, flags escalations by deterministic rule, and keeps eTMF records human + validated. No date or escalation threshold is computed by the model. Binding constraint is extraction recall — field/action-item extraction from short passages is the SLM's reliable zone, but a missed or hallucinated action item is the failure mode, so SAS/R owns every date computation and the human gate confirms each item before it counts.
QTL dashboard upkeep & RBQM oversight memo	⚠️ Guardrails	**SLM:** Draft per-item change-log entries, short single-trend annotations, and recalibration flags grounded in supplied values — explicitly NOT the synthesized whole-portfolio oversight memo, which is held back from the model **SAS/R:** The validated platform produces all dashboard values; SAS/R computes every trend delta and amendment-triggered recalibration condition, validates that annotations cite only supplied numbers, and assembles the periodic oversight summary structure deterministically before the RBQM lead reviews. The model never computes a trend or synthesizes the multi-KRI picture. Binding constraint is that the periodic oversight memo aggregates many KRI/QTL movements into one narrative — multi-signal synthesis that edges toward Beyond. The component stays Guardrails only by amputating that synthesis to SAS/R and limiting the model to per-item annotations; if it were allowed to write the whole oversight picture unaided it would be Beyond.

The platform layer

These are infrastructure, not model tasks — and the on-device variant *simplifies* most of them: one offline local model means there is no cloud egress to firewall and no model-agnostic gateway to run; SAS/R calls a single local endpoint, pins the model, and writes the audit trail.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
Model-agnostic gateway (LiteLLM-class) — single OpenAI-compatible front door	◇ Platform	**SLM:** none — this is the routing/control plane that CALLS the SLM; it performs no language task itself **SAS/R:** SAS/R is effectively the gateway: PROC HTTP / httr2 posts to ONE local 127.0.0.1 endpoint, applies the data-classification routing rule, enforces budgets/rate limits, retries/falls back, and writes one structured log line per call; SAS/R owns any counting/metering math, never the SLM Pure routing/cost/audit choke point, not a language task; binding constraint is that with a single offline local SLM the 'model-agnostic' abstraction collapses to 'SAS/R calls one fixed local 127.0.0.1 endpoint' — the multi-backend routing logic mostly disappears and the value reduces to a single auditable call site.
Data-classification router + Presidio PII/PHI guardrail (egress firewall)	◇ Platform	**SLM:** none — classification/redaction is deterministic Presidio NER+regex; a hallucination-prone SLM must NEVER be the thing deciding what counts as PHI **SAS/R:** SAS/R drives Presidio recognizers, applies the allow/deny routing policy, masks/blocks misclassified-sensitive text, logs the routing decision, and samples false-negatives for the privacy audit; SAS/R owns all match/count tallies Egress-firewall infrastructure (the framework's named Platform example), not language work; binding constraint is that a single PHI false-negative leaks data so it stays deterministic — and in a strictly-offline single-SLM build there is no cloud egress to firewall at all, the control degenerates to 'the box stays offline,' making Presidio a defense-in-depth/audit layer rather than the live gate.
LOCAL frozen open-weight backend (vLLM, air-gapped GPU)	◇ Platform	**SLM:** none — this IS the model-server box; in the on-device variant it is Ollama/llama.cpp/LM Studio serving a 4-8bit quantized 1-9B GGUF on an ordinary workstation, not a vLLM GPU farm **SAS/R:** SAS/R is the only caller and pins model name+quantization+digest, temperature-0/fixed-seed decoding, and the prompt as a frozen re-runnable artifact; SAS/R keeps the box air-gapped and NEVER lets the model emit a reported/regulated number — all numbers come from validated engines Backend infrastructure, not a language task; binding constraint is capability — a quantized 7-9B local model is the realistic on-device backend, so every downstream language verdict inherits its HIGHER hallucination rate and prompt/format sensitivity, and 'frozen + air-gapped' is exactly what buys reproducibility to offset that ceiling.
CLOUD Claude backend (API under BAA, de-identified only)	◇ Platform	**SLM:** none — and explicitly OUT OF SCOPE for an on-device SLM; this is the frontier escalation path, not a small local model **SAS/R:** if kept, SAS/R routes only de-identified text here after the classifier clears it and queues the draft for human accept/reject; in a strictly offline build SAS/R simply has no route to this backend Not an SLM Platform at all — it is the larger-model escalation target named in the 'Beyond' guidance; binding constraint is that the on-device-SLM thesis must either DROP this entirely (fully offline) or reserve it explicitly as the escalation for hard reasoning/long-context synthesis a 7-9B quantized local model cannot do reliably; it cannot count as 'realized on-device.'
RAG knowledge base over SOPs/WIs/study	⚠ Guardrails	**SLM:** from a SINGLE retrieved SOP/WI passage, produce a short extractive/templated answer to one narrow compliance question, copying the

docs (local, permissioned vector store)		citation (doc id + version) verbatim from the retrieved chunk's metadata — no cross-passage reasoning, no paraphrase of obligations beyond the cited text **SAS/R:** SAS/R owns ingestion/chunking/embedding into the local store, permissioned retrieval, and the deterministic citation-verification validator (every cited doc+version MUST exist in the retrieved set and the quoted span must match, else the answer is rejected); SAS/R surfaces the controlled document as authority and counts/ranks retrieval scores — never the SLM The ONLY genuine SLM language sub-task in this area, and it sits at the EDGE of the reliable zone; binding constraint is multi-document synthesis — quantized 7-9B models drift and conflate obligations when asked to combine several SOP chunks or reason about long context, so it is safe ONLY single-passage, extractive, citation-constrained, validator-checked, with the controlled document (not the model) as authority and a human verifying the citation; widen scope to multi-SOP synthesis or interpretive 'does this comply?' judgement and it becomes Beyond.
Orchestration layer (scheduled jobs + agent framework that fires the agents)	◇ Platform	**SLM:** none — scheduling, chaining, timeouts, and queue-interception are deterministic orchestration code, not language work **SAS/R:** SAS/R IS the orchestrator: cron/batch schedules jobs on study events, sequences local-endpoint calls with connector tool-calls, enforces per-agent timeouts/guardrails, and drops any record-of-truth action into the approval queue instead of executing it; SAS/R owns all control-flow logic and any timing/retry counters Pure automation harness, not a language task; binding constraint is that orchestration must be fully deterministic and auditable — exactly what SAS/R scheduling provides — and no SLM belongs anywhere in the control flow, only as a constrained sidecar call SAS/R makes for a single language sub-task.
Connector/tool layer via MCP (eTMF, Smartsheet, EDC, CTMS, SharePoint, Outlook, Pinnacle 21)	◇ Platform	**SLM:** none — connectors broker and log system access; the language work lives in the agents that call them, not in the connector layer **SAS/R:** SAS/R reaches each system with least-privilege scopes (PROC HTTP/httr2/native libraries), keeps reads read-only, PROPOSES every record-of-truth write to the approval queue rather than executing, and logs each access; SAS/R owns any reconciliation/diff computation feeding a proposed write Access-brokering infrastructure, not a language task; binding constraint is least-privilege + write-gating, which SAS/R native connectors enforce directly — in an on-device build the MCP abstraction often collapses into plain SAS/R API calls, and the hard line is that NO write to a record of truth executes without the human-gated approval queue.
Unified Part 11 audit & observability trail	◇ Platform	**SLM:** none — recording who/what/when/before-after is logging infrastructure, never a generative task **SAS/R:** SAS/R writes every call's record (agent, pinned model version+quantization+digest, prompt, classification decision, redactions, tokens/cost, tool calls, human e-signature, before/after of any record change) to one tamper-evident time-stamped store and never edits or deletes entries Compliance infrastructure that makes the stack inspectable, not a language task; binding constraint is tamper-evidence and completeness — an SLM has no role and must NEVER be allowed to author, summarize, or alter trail entries (a hallucinated audit line is worse than none); the offline single-model build simplifies it to logging one local endpoint plus connectors.
Human-in-the-loop approval queue	◇ Platform	**SLM:** none — the queue presents the draft, diff, and citations; the human decides and signs **SAS/R:** SAS/R holds every proposed regulated change as a reviewable item, renders the AI draft + source/citations + diff + classification, BLOCKS the

		connector until an explicit human accept/edit/reject + e-signature, then executes and logs; SAS/R computes the diff deterministically This is THE human-gate mechanism the whole framework depends on, not a language task; binding constraint is that no SLM output reaches a record of truth without a named reviewer's signature — it is the safeguard that makes every Guardrails verdict in the program tolerable, and it tightens, not loosens, with a higher-hallucination on-device model.
Model-freeze & engine-of-record registry	◇ Platform	**SLM:** none — and the registry's whole purpose is to EXCLUDE the SLM from ever being an engine of record **SAS/R:** SAS/R consults the registry to pin which frozen local model version+quantization+digest+decoding+prompt is in production per task (reproducibility) and which VALIDATED tool is the engine of record for each regulated output (Pinnacle 21 = conformance, Phoenix WinNonlin = PK, EDC/CTMS = operational truth); SAS/R refuses to treat any SLM output as a reported number The framework's explicit Platform example, not a language task; binding constraint is the hard line that reported/regulated numbers come ONLY from validated engines — the registry exists precisely so a quantized SLM can never become one, and freezing model+quant+digest+seed+prompt is what makes an on-device run re-executable identically for an inspector years later.
Caching, cost/budget controls & FinOps metering	◇ Platform	**SLM:** none — caching and metering are deterministic gateway features, not language work **SAS/R:** SAS/R caches identical/near-identical de-identified prompts (classification-aware; sensitive payloads excluded), meters per agent/model/study/team, and enforces budget caps; SAS/R owns all cost/token arithmetic Operational-efficiency infrastructure, not a language task; binding constraint is largely MOOT on-device — a single offline local model is fixed-cost hardware with no per-token cloud bill, so FinOps shrinks from runaway-spend control to capacity/throughput/latency tracking (GPU-hours, queue depth) on the one box.
Identity, RBAC & secrets / trust-boundary management	◇ Platform	**SLM:** none — authentication, authorization, secret-holding, and host isolation are security control-plane functions, not language tasks **SAS/R:** SAS/R (with the security stack) maps users/agents to roles/studies, enforces least-privilege on local-endpoint keys and connector scopes, pulls credentials from a vault, and keeps the air-gapped local host isolated; no number or decision here originates from an SLM Security infrastructure underpinning every RED safeguard, not a language task; binding constraint is keeping the air gap real and permissions per-study — an SLM makes no decision here, and the offline single-box deployment is exactly what makes 'the air gap real' physically enforceable rather than a policy promise.

Additional / easily-missed

The easily-missed ops: consent-version tracking, IP accountability, safety-clock and submission tracking, training compliance, data-transfer receipt. Extraction and reminders suit a small model; the regulated records stay validated + human.

Component	On-device?	What the small model does · what SAS/R owns · the honest limit
Informed-consent (ICF) version tracking & site-version reconciliation	⚠️ Guardrails	**SLM:** Extract verbatim version IDs, language/translation labels, and printed effective/approval dates from each ICF header/footer or approval stamp (short, located passages) into a structured row as candidate values for human confirmation. The SLM does NOT classify the match/superseded/re-consent-due bucket — that is a date computation, not language. **SAS/R:** Owns the authoritative version-effective-date table, the consent-date-vs-effective-window comparison and ALL bucketing logic (match / signed-superseded / re-consent-due / missing-site-approval), re-consent triggering, the validator that checks SLM-extracted dates against allowed formats/ranges, every write, and the audit log; runs air-gapped because consent dates are PHI-adjacent/unblinded. Binding constraint is stakes plus a hidden numeric step: consent-version mismatch is a top-5 GCP finding, so the date/window comparison must be deterministic SAS/R — the draft let the SLM 'classify into a bucket,' which is the date logic in disguise; pull it back to extraction-only behind a PM/CRA gate.
Site feasibility, selection & activation-readiness tracking	⚠️ Guardrails	**SLM:** Draft feasibility questionnaires from a template and extract/normalize individual fields from EACH single returned site response (capacity, equipment, prior-experience flags) into a structured row; optionally a one-line per-site status note. One response at a time, never cross-site. **SAS/R:** Owns the activation checklist state, per-site cycle-time and status computation, assembly of the ranking matrix from extracted fields, the activation-slip prediction model, scheduling/alerts, schema validation, writes, and audit log. Binding constraint is multi-response synthesis and prediction: rolling many free-text feasibility replies into a faithful ranking and forecasting slips is multi-document/quantitative work a small model cannot do reliably, so SAS/R must assemble and score and the SLM is confined to per-response field extraction.
Investigational-product (IP)/drug accountability & reconciliation	✅ Ready	**SLM:** Draft the accountability-discrepancy memo for CRA follow-up from SAS/R-supplied, already-computed discrepancy figures (templated, grounded drafting only) — the SLM inserts no numbers of its own. **SAS/R:** Owns the entire reconciliation (shipped vs dispensed vs returned vs destroyed) against IXRS/IRT and accountability logs, negative-inventory/expiry/temperature-excursion/missing-destruction detection, and the count of record from the validated IXRS/inventory system; air-gapped, writes and audit log included. Binding constraint is numbers, and they never touch the SLM: every count comes from validated systems, leaving the SLM a clean templated-drafting task behind a CRA/QA signature — a genuine Ready.
Randomization / IXRS(IRT) setup-spec & UAT oversight (blinding-safe)	⚠️ Guardrails	**SLM:** Convert UAT outcomes into structured defect-log entries and candidate test-step text for human review, and extract individual protocol-stated parameters (named stratification factors, stated block size, dispensing rules) into a structured spec-input row. The SLM does NOT author the multi-section randomization spec narrative and does NOT QC spec-vs-protocol fidelity — that reasoning is escalated. **SAS/R:** Owns protocol-parameter extraction validation, assembly of the spec

		document from validated fields/templates, UAT execution tracking, live stratification-imbalance/assignment-anomaly statistics computed without exposing the randomization list, schema validation, writes, and audit log; strictly air-gapped. Binding constraint is spec-fidelity reasoning plus unblinding sensitivity: authoring and QCing a randomization spec (block/strata/unblinding logic) genuinely edges past a small quantized model, so that sub-task escalates to a larger local model or stays human-authored while the SLM is held to UAT structuring and per-field extraction under an unblinded-statistician/QA gate.
Lab manual, central-lab spec & kit/supply management	⚠️ **Guardrails**	**SLM:** Extract and normalize assessment names from the protocol schedule-of-assessments and the analyte/panel/kit-content names from lab requisitions into a controlled-vocabulary structured list; draft templated resupply requests over SAS/R-supplied quantities. **SAS/R:** Owns the set-difference coverage check (protocol vs lab spec), per-site inventory and expiry tracking, resupply forecasting and all quantities, the validator over normalized names, scheduling/alerts, writes, and audit log. Binding constraint is cross-document coverage reasoning: gap detection must be a deterministic SAS/R set comparison over SLM-normalized terms, because letting the model 'decide' coverage invites missed-panel hallucinations on a classic deviation source.
Regulatory submission & lifecycle tracking (IND/CTA, amendments, annual reports)	✅ **Ready**	**SLM:** Draft templated cover letters / 1571-class form text and assemble the submission-component checklist from a retrieved template (grounded, narrow drafting); optionally a one-line deadline reminder. No date arithmetic. **SAS/R:** Owns the submission calendar and all agency-clock/anniversary date arithmetic (30-day safety-to-proceed, DSUR/IND-annual-report anniversaries), deadline alerting, checklist-completeness logic, writes, and audit log; the official record stays the regulatory system of record. Binding constraint is the recurring clock math, which is pure SAS/R date arithmetic, leaving the SLM only template-fill drafting behind a regulatory-affairs signature.
Expedited safety-reporting clock tracker (SUSAR/SAE 7- & 15-day)	✅ **Ready**	**SLM:** Draft distribution-log entries and pre-deadline reminder notes from SAS/R-supplied case IDs, recipients, and clock status (templated drafting only); explicitly does NOT draft or judge the medical/causality assessment. **SAS/R:** Owns the 7/15-day clock start from sponsor-awareness date and all clock/follow-up arithmetic, recipient-distribution tracking, at-risk-case flagging, writes, and audit log; air-gapped (case safety data is PHI/unblinded). Binding constraint is severe regulatory stakes, mitigated because causality/expectedness and the official submission stay with PV/medical-monitor and SAS/R owns every clock, so the SLM's reminder/log drafting is low-residual-risk.
Audit & inspection logistics / readiness orchestration	⚠️ **Guardrails**	**SLM:** Draft the document-request list and war-room/agenda tracker entries from templates and tag incoming inspection requests to the right SME/queue (routing/triage). The SLM does NOT draft audit-observation response content — not even a 'skeleton' — because narrative response substance is escalated or human-authored. **SAS/R:** Owns the logistics tracker state, SME-availability and finding-to-CAPA-closure tracking, request logging, routing validation, scheduling, writes, and audit log. Binding constraint is the audit-response narrative: composing (or even skeletoning) a defensible regulatory response is nuanced and high-stakes, genuinely beyond a small model and able to plant wrong commitments, so that sub-task escalates to a larger model or stays human-authored while the SLM is confined to logistics tagging under a QA gate.

Training-compliance & qualification tracking (curriculum + delegation)	✅ **Ready**	**SLM:** Draft the training-status summary for the eTMF from SAS/R-computed compliance results (templated drafting); optionally normalize free-text course/role names into the controlled training-matrix vocabulary. **SAS/R:** Owns the required-training matrix, LMS-completion-vs-assignment reconciliation, the training-to-delegation linkage logic, overdue/expired-cert and delegation-without-prerequisite detection, validation, alerts, writes, and audit log. Binding constraint is reconciliation logic, which is deterministic SAS/R, so the linkage that catches untrained-but-delegated staff never depends on the model and the SLM only summarizes/normalizes.
Data-transfer agreement (DTA) scheduling & receipt/reconciliation confirmation	✅ **Ready**	**SLM:** Draft the transfer-receipt log entries and late/missing-transfer notices from SAS/R-supplied arrival status and validation results (templated drafting only). **SAS/R:** Owns the transfer calendar derived from each DTA spec, arrival confirmation, file-structure/record-count/checksum validation against spec, count reconciliation against EDC, flagging, writes, and audit log; air-gapped. Binding constraint is numeric/structural validation (counts, checksums), entirely SAS/R, so the SLM is confined to drafting the log over already-validated facts.
Contract / CDA / agreement lifecycle tracking (CTA-contract, CDA, MSA, vendor SOWs)	⚠️ **Guardrails**	**SLM:** Over RAG-retrieved short chunks only, extract key fields and obligation/clause snippets (effective/expiry dates, auto-renewal and notice windows, named deliverables, payment triggers) as candidate values for mandatory legal confirmation — every extracted clause is treated as unverified. **SAS/R:** Owns the agreement portfolio state, all date/notice-window arithmetic and expiry/renewal alerting, the unexecuted-agreement-blocking-activation logic, the validator over extracted fields, writes, and audit log. Binding constraint is clause extraction from long, nuanced legal text that runs past the reliable short-passage window — this sits right at the Beyond boundary, so it survives as Guardrails only because RAG chunks the input and legal human-verifies every clause while all date logic stays in SAS/R.
Lessons-learned & continuous-improvement knowledge capture	❌ **Beyond**	**SLM:** Per single artifact, can extract a structured issue record and propose one near-duplicate cluster label; but the headline deliverable — faithful thematic synthesis across many closed studies, CAPAs, deviation/query trends, and minutes — is not reliable on a small quantized model. **SAS/R:** Owns retrieval/RAG indexing, embedding-based similarity grouping of issue records, trend statistics, surfacing prior lessons at study start-up, writes, and audit log. Binding constraint is multi-document synthesis: clustering short records is feasible but cross-artifact thematic synthesis needs frontier-level reasoning, so escalate the synthesis step to a larger local/cloud model (de-identified) while SAS/R does retrieval and grouping and a QA/process owner curates before anything becomes guidance.
Sponsor-update / newsletter & external communication drafting	✅ **Ready**	**SLM:** Draft the recurring sponsor update / newsletter by filling the house template with SAS/R-supplied enrollment metrics, milestone status, and safety headlines, producing tailored sponsor-vs-site versions (templated drafting grounded in retrieved values). **SAS/R:** Owns metric/milestone retrieval from CTMS/dashboards, the de-identification/no-unblinded-data egress check, distribution-log maintenance, review routing, writes, and audit log. Binding constraint is keeping unblinded figures out of an external draft, enforced by a SAS/R egress/de-id check plus a PM send-gate, leaving the SLM a clean template-fill task it does well.
Participant travel, reimbursement & visit-	✅ **Ready**	**SLM:** Draft participant visit reminders and reschedule/conflict notices from SAS/R-computed schedules and window-risk flags (templated drafting only).

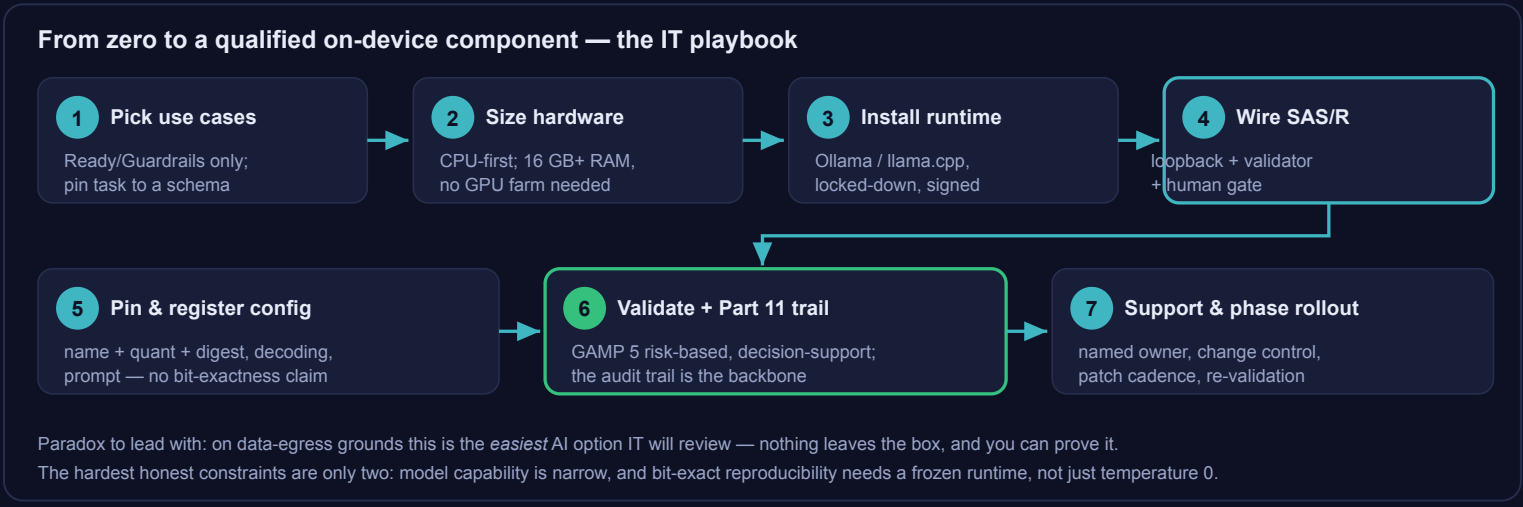
logistics coordination (early-phase unit ops)		**SAS/R:** Owns confinement/ambulatory PK scheduling against strict protocol time windows, bed/slot capacity tracking, window-risk and overlapping-cohort conflict detection, stipend/reimbursement workflow state, writes, and audit log; air-gapped (participant identity/travel/payment is PHI). Binding constraint is the rigid PK-window/capacity math, which is deterministic SAS/R scheduling, so the SLM never computes timing and only drafts the human-facing reminders behind a unit-coordinator gate.
eCOA / ePRO / wearable-device data oversight & compliance monitoring	✅ Ready	**SLM:** Draft site-level compliance alert text from SAS/R-computed completeness/compliance results (templated drafting); does NOT adjudicate compliance status. **SAS/R:** Owns the completeness/wear-time/sync-gap statistics, implausible/curbstoned-pattern detection, reconciliation of eCOA records against the expected diary/visit schedule, flagging for central monitoring, writes, and audit log; air-gapped (participant-level PHI). Binding constraint is the integrity/pattern statistics, all SAS/R, with the SLM limited to drafting alerts over flags that DM/central-monitoring adjudicates, keeping the model out of any compliance judgement.

Standing it up with IT — instructions & hurdles

This is the section to hand IT, Security, and QA. It is two parts: a concrete, numbered **enablement playbook** for deploying an on-device small model alongside SAS/R, and a candid **hurdles table** naming what each function will raise and exactly how to clear it. Throughout, hold the line: **SAS/R and the validated tools (Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS) own every number and every record. The small model is a constrained, offline language helper behind a human gate.** Nothing here asks IT to trust an AI with a regulated value.

> **One paradox to lead with.** On data-governance grounds this is the *easiest* AI option IT will ever review — the model is offline on one box, so there is no egress to firewall, no vendor BAA to chase, no telemetry to disprove. You can demonstrate the security control by pulling the network cable. The *hardest* honest constraint is the opposite of the usual one: not "can we secure it" but "is the small model actually good enough" — which is why scope is everything. Do not let the easy security story tempt anyone into a hard language task.

Part 1 — the enablement playbook



The enablement playbook at a glance: seven steps from picking Ready/Guardrails use cases through CPU-first sizing, runtime install, wiring SAS/R with a validator and human gate, pinning and registering the config, GAMP 5 validation with the **Part 11 audit trail as the backbone**, to a supported, change-controlled rollout.

1. **Pick the use cases and pin the language task to a schema.** Choose only from the `Ready` (green) and `Guardrails` (amber) components — short classification, field extraction, routing/triage tags, near-duplicate grouping, or a short RAG-grounded draft. For each, write down the *exact* language sub-task and the **output schema** (the allowed labels/enum, the fields, the JSON shape). If you cannot reduce it to a fixed-vocabulary, short-context decision, it is `Beyond` a small model — leave it deterministic SAS/R or escalate to a larger model; do not force it on-device.
2. **Size the hardware — CPU-first.** A modern workstation runs a 7–8B model quantized to Q4 (e.g. Q4_K_M GGUF) at a usable, non-interactive speed on CPU alone; 3–4B models are comfortably faster. Memory rule of thumb: at Q4 the *weights* are roughly *0.5 GB per billion parameters* (~4–5 GB for an 8B), but you must add the KV cache, the context, and runtime overhead — so budget closer to **~6 GB resident for an 8B Q4 model and provision 16 GB+ of system RAM** to run it comfortably alongside SAS/R. Add **one consumer GPU only where throughput demands it** — for triage-style batch jobs that run nightly or in small bursts, CPU is usually enough. A single 12–16 GB GPU fits an 8B model in VRAM and multiplies tokens/sec if a queue ever justifies it. (Note: moving from CPU to GPU can change exact byte-for-byte outputs — see step 5 on reproducibility.)

3. **Choose a runtime and install it in the locked-down environment.** `Ollama` for the simplest operations (one local service, easy model management); `llama.cpp` when you want maximum control or to embed the engine (Ollama wraps llama.cpp, so they share the GGUF runtime); `LM Studio` for desktop trials and quick evaluation; `vLLM` only later if you genuinely need server-grade throughput on a GPU host. Install from a **signed release pinned to a version**, mirrored on the internal artifact repository (or delivered as an approved container image). Pull the chosen model on a connected *staging* box, record and **checksum the model file (SHA-256 digest)**, then transfer that verified file to the offline workstation — the production box never reaches the internet.
4. **Wire SAS/R to the local endpoint and bolt on the validator + human gate.** SAS calls the loopback endpoint with `PROC HTTP`; R uses `httr2` — both target `127.0.0.1` only. SAS/R does all data access and computation, then sends just the short language sub-task. The model returns **schema-constrained JSON** (enforced with the runtime's grammar / JSON-schema / structured-output mode, not just asked for in the prompt); a **SAS/R validator** then parses it and checks every field against the allowed values, rejecting and logging anything malformed or off-list. Nothing reaches a record of truth until a human reviews and approves it.
5. **Pin the configuration for best-effort reproducibility; register it; do not claim bit-exactness.** Freeze *model name + quantization + SHA-256 digest*, the **decoding settings** (temperature 0 / greedy, top_k and top_p pinned, repeat/frequency penalties pinned, a fixed seed for any stochastic path), and the **full runtime context**: runtime build/version, hardware, thread count, and single-stream **batch = 1** execution. Be honest with QA about what this buys you: temperature 0 makes the *token-selection rule* deterministic, but it does **not** guarantee byte-identical output across different hardware or under varying batch/load, because floating-point addition is non-associative and kernels accumulate in different orders. Pinning the runtime, the hardware, and batch = 1 (CPU single-stream is the most reproducible path) gets you run-to-run stable output *on that box*; treat cross-hardware bit-exactness as **not assured**. Enter the frozen tuple in the engine-of-record / model-freeze registry exactly as a version of a qualified tool — no silent model, quant, runtime, or hardware swaps.
6. **Validate (GAMP 5 Second Edition, risk-based) and wire the Part 11 audit trail — which is the real backbone.** Scope the model as low-to-medium-risk *decision-support*, not a system of record, and follow GAMP 5 (2nd ed.) with the ISPE GAMP *RDI Good Practice Guide, Appendix D11 (AI/ML)*: treat it as a probabilistic component, not deterministic software. **IQ** the install (runtime version, model digest, offline state, batch = 1 config). **OQ** the pinned model against a fixed, labeled test set — measure label accuracy and JSON-schema adherence, set an acceptance threshold, and include a **re-run stability check** (same inputs run repeatedly on the production box) rather than assuming one golden output. **PQ** on the real workflow with humans in the loop. Because exact reproduction is configuration-dependent, the **21 CFR Part 11 audit trail is what makes this defensible**: SAS/R writes, for every item, *prompt + model digest + runtime/config + raw model output as actually returned + the validator result + the human decision + timestamp + user.* You audit the output that *actually occurred* and who approved it — you do not rely on being able to regenerate it bit-for-bit.
7. **Define the support model and phase the rollout.** Name an owner; treat the model like a qualified instrument under change control (any change to model, quantization, runtime build, or the hardware/decoding config is a controlled change with re-validation). Set a patch cadence for the runtime. Roll out in phases: one narrow `Ready` use case on one box first, prove the OQ/PQ and the audit trail, then widen to the next.

Part 2 — anticipated hurdles

What IT, Security, and QA will raise — and the honest answer for each. None of these requires overselling the model.

Hurdle (what they'll raise)	Why it comes up	How to clear it
"We don't allow unknown binaries or downloads."	Hardened endpoints block arbitrary executables and	Use a **signed, version-pinned release** from the internal artifact mirror or an approved container image. **Air-gap the model transfer**: pull on a staging

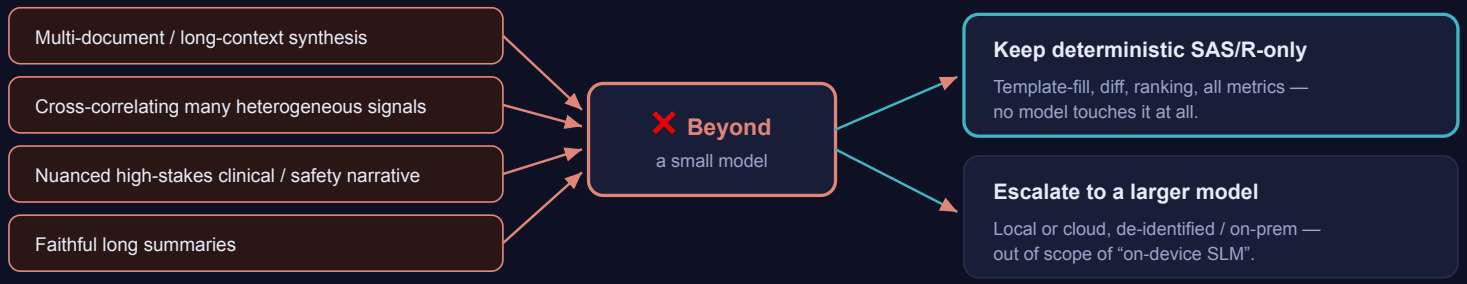
	internet pulls; the runtime and model are both new artifacts.	box, checksum (SHA-256), move the verified file to the offline box. Nothing is pulled live on production.
"We have no GPU budget."	People assume "AI" means a GPU farm.	It doesn't here. **CPU-first**: a small quantized model (3–8B, Q4) runs at usable speed on a normal workstation with ~6 GB resident and 16 GB+ system RAM. Add a single consumer GPU only on the specific component where throughput actually demands it.
"Is the AI validated? Is it deterministic?"	QA cannot accept a non-reproducible component in a GxP path.	Be precise, not promotional. **Frozen model** (name+quant+digest) + **temperature 0** + pinned runtime/hardware + **batch = 1** gives run-to-run stable output on the box, but byte-exactness is *not* guaranteed across hardware (floating-point non-associativity). So the controls are: **GAMP 5 2nd ed. + Appendix D11 risk-based CSV** (IQ/OQ/PQ with a re-run stability check) qualifying it as *decision-support*; a **SAS/R validator** on every output; a **human gate** before any record; and a **Part 11 audit line capturing the actual output and the human decision**. The SLM is never the record — SAS/R and the validated tools are.
"Where does the data go?"	The default fear with any AI is data leaving to a vendor.	**Nowhere.** The model is offline on the device; SAS/R only ever calls `127.0.0.1` and asserts the loopback before use. Demonstrate it by pulling the network cable — the workflow still runs. This is the **simplest data-governance story of any AI option**: no egress, no vendor, no telemetry.
"What about endpoint / device security?"	A workstation now holds study text and runs an inference service.	Treat the box as a **controlled instrument**: full-disk encryption, standard endpoint management, least-privilege access, the inference service bound to loopback only. An offline box has **no outbound channel** to exfiltrate through; harden it like any other instrument holding study data.
"What about the open-weight model's licence?"	Legal must confirm commercial use and provenance before it ships, and open-weight licences vary *by size within the same family*.	Prefer genuinely permissive weights — **IBM Granite 4.1 (Apache-2.0 across 3B/8B)**, **Qwen3 or Gemma 4 (Apache-2.0)**, or **Phi-4-mini (MIT)**. Watch the traps across releases: *Qwen consolidated to uniform Apache-2.0 in Qwen3* (the older Qwen2.5 split its 3B/72B under a custom Qwen licence), and *Gemma moved from a custom "Gemma Terms of Use" in Gemma 2/3 to Apache-2.0 in Gemma 4*. Legal reviews the **specific model and size variant**, its licence, and its provenance/safety documentation as a gating step — recorded with the registry entry, not waved through.
"Who supports and patches it?"	Ungoverned shadow tooling is an audit finding.	**Named owner**, a documented patch cadence for the runtime, and the model under a **change-controlled freeze registry**. Any change to the model, quantization, runtime build, or decoding/hardware config is a controlled change requiring re-validation — exactly like a version bump on a qualified analytical tool.
"Small models hallucinate — how do we trust the output?"	Correct instinct; SLMs hallucinate more and are more prompt- and format-sensitive than frontier models.	The risk is **bounded by design**: grammar/schema-constrained output + RAG grounding on a short retrieved passage + a **SAS/R validator** that rejects anything off the allowlist + a **human gate**. And critically, **the model never produces a number** — all counts, checks, reconciliation, and stats are SAS/R's and the validated engines'.
"Will it keep up at volume?"	A small model on CPU has finite throughput.	**Batch and schedule off-hours** (these jobs are nightly, not interactive), **right-size the model** (use the smallest one that passes OQ), and only then **scale to a single GPU host** if a real queue justifies it — re-validating, since a hardware change can shift exact outputs.

The honest bottom line for IT. Paradoxically, the on-device SLM is the *easiest* AI option to approve on data-egress and privacy grounds — nothing leaves the box, and you can prove it physically. The *hardest* honest constraints are two: model capability (a small quantized model is a narrow language helper, not a reasoning engine) and the fact that "deterministic" is a *configuration-pinned, best-effort* property, not a guarantee of byte-exact reproduction across hardware. Both are handled by the same discipline: keep the language work short, bounded, schema-constrained, and validated; pin model + quant + runtime + hardware + decoding and register it; capture the *actual* output and the human decision in the Part 11 audit trail; leave every number to SAS/R and the validated engines; and send anything needing real synthesis or nuance to a larger model or keep it deterministic. Scope and pin honestly and this is a quick yes; oversell the model's reasoning or its determinism and it should be a no.

What a small on-device model can't do — and why

The 15 components marked **Beyond** are the honest line: the *language* work needs frontier-level reasoning, long-context synthesis, or nuanced clinical/regulatory narrative that a small quantized model will not do reliably. Keep these **deterministic SAS/R** or escalate to a larger local/cloud model.

Why a component is "Beyond" — and where it goes instead



In every case SAS/R still owns all numbers, the diff, the ranking, and the assembled skeleton; only the narrative step is at issue.

A small on-device model is never the answer here — it fails fluently, which at lock / safety / inspection stakes is worse than failing loudly.

15 components land here: status reports · portfolio rollup · KRI/QTL · CSM signals · safety triage · DSMB packets · pre-lock review · lessons-learned · more.

The Beyond line: a component is Beyond when its language work needs long-context synthesis, multi-signal correlation, nuanced high-stakes narrative, or a faithful long summary — and from there it is either kept **deterministic SAS/R-only** or **escalated to a larger model** on de-identified data, never handed to an on-device small model.

✕ Status-report generation (study & program)

Binding constraint: faithful multi-document synthesis into a long report is exactly where small quantized models degrade and fabricate — keep template-fill deterministic SAS/R and escalate any cross-section narrative to a larger local/cloud model, not an on-device SLM. *SAS/R owns:* Pull every metric from systems of record, fill the house-template slots deterministically, flag any RED metric for human verification against the source, assemble the report skeleton, and own all numbers — none are ever produced or transformed by the SLM.

✕ Cross-study portfolio rollup

Binding constraint: many-document, long-context cross-correlation is the precise degrade zone for small quantized models — keep the rollup and exception ranking deterministic SAS/R and escalate any portfolio narrative to a larger local/cloud model. *SAS/R owns:* Aggregate cross-sheet metrics from every study's system of record, compute slip/risk/budget health, rank and select the exceptions deterministically, and build the dashboard table; the cross-study narrative is what exceeds the SLM.

✕ Milestone-slip root-cause & schedule-recovery proposal

Binding constraint: root-cause correlation across many heterogeneous signals plus scenario reasoning is genuine frontier-level work — keep the impact math in Smartsheet and escalate the diagnosis/recovery narrative to a larger local or cloud model, not an on-device SLM. *SAS/R owns:* Confirm the slip, gather the linked actions/risks/deliverables/notes, run all recovery impact modeling in Smartsheet (fast-track/re-sequence/resource-add timelines), and hold scenario selection for the PM/lead.

✕ Inspection-readiness gap analysis

This cross-correlates many heterogeneous signals into a nuanced, high-stakes readiness judgement and executive narrative — long-context multi-signal synthesis small models do unreliably; the binding constraint is exactly that synthesis, so keep the scan deterministic SAS/R-only and ESCALATE any generated narrative to a larger local or cloud model on de-identified data. *SAS/R owns:* Computes every input signal (zone completeness, overdue/placeholder, unsigned/out-of-window, expiring credentials, superseded references, inspector hot-spot rules) and assembles the prioritized remediation plan with owners deterministically

✕ TMF audit-trail & filing-activity summarizer

This is faithful long-summary and multi-signal synthesis over many log records — exactly where small models lose fidelity and invent trends; the binding constraint is faithful long-context aggregation, so keep aggregation deterministic SAS/R-only and ESCALATE the executive narrative to a larger local or cloud model on de-identified data, with the TMF lead reviewing before governance. *SAS/R owns:* Reads the immutable Part-11 audit trail, computes ALL metrics (filing volumes, backlog age, who-filed-what, recurring rejection reasons, bottleneck trends) deterministically, and never edits the trail; reported numbers tie back to the system of record

✗ Query-management oversight, aging & SLA escalation

Binding constraint is FORBIDDEN MATH plus multi-query synthesis: as written the component hands the SLM aging/SLA/lock-threat computation and landscape summarization, so keep ALL analytics DETERMINISTIC SAS/R-only (escalate any cross-query narrative to a larger model) and demote the SLM to at most a single per-item email draft. *SAS/R owns:* Pulls open-query metadata and deterministically computes aging buckets, SLA-breach flags, milestone-risk ranking, and the prioritized worklist; the SLM is given finished outputs only.

✗ Data-quality KRI / QTL trend dashboard upkeep (RBQM)

Binding constraint is LONG-CONTEXT MULTI-KRI SYNTHESIS with signal-vs-noise judgement — exactly what a quantized 7-9B model degrades on — so ESCALATE the decision-grade narrative to a larger local or cloud model; on-device, the SLM is limited to single-metric captions. *SAS/R owns:* Computes all KRI/QTL values, breach status against ICH E6(R3) acceptable ranges, and which metric moved; owns the validated metric layer driving every threshold.

✗ Statistical / central-monitoring anomaly signals (site outliers, enrollment & visit anomalies)

Binding constraint is HIGH-STAKES MULTI-SIGNAL CORRELATION on fraud-adjacent signals — cross-KRI integrity-vs-operations adjudication is beyond a small quantized model — so ESCALATE that synthesis to a larger model and restrict the on-device SLM to one memo per already-ranked signal behind a central-monitor gate. *SAS/R owns:* Runs the validated CSM engine computing site-vs-site outlier statistics, variance/mean anomalies, enrollment/visit-pattern and over-uniformity signals, and the ranked signal list; supplies de-identified summaries only.

✗ Pre-lock data-review listing generation, diff & sign-off packet

Binding constraint is LONG-CONTEXT SYNTHESIS AT LOCK STAKES: faithful multi-listing diff summarization and lock-readiness/SAP-gap judgement are beyond a small model, so keep the diff and completeness checks DETERMINISTIC SAS/R-only, ESCALATE any prose summary to a larger model, and the biostatistician owns sign-off. *SAS/R owns:* Generates the validated blinded listings, computes the diff against the prior review run, maps open items to queries/dispositions, and deterministically checks which SAP-required checks are evidenced.

✗ KRI/QTL definition & specification drafting

Binding constraint is multi-document synthesis plus regulated CtQ judgement across protocol+risk-assessment+SOPs — long-context reasoning that a 7B quantized model degrades on well before its context window, with high adjudication stakes (a wrong KRI definition mis-aims the whole monitoring plan). Keep only deterministic/templated catalog scaffolding local; the real drafting is Beyond on-device. *SAS/R owns:* RAG retrieval over the protocol/SOP corpus; the validated CSM platform computes every metric and every threshold of record; SAS/R logs provenance and routes any escalated draft to the risk committee and biostat lead. No number, threshold, or definition of record originates in any model.

✗ Central statistical monitoring signal interpretation (site outliers, digit preference, anomalies)

Binding constraint is multi-signal correlation into a coherent fraud/integrity narrative with high adjudication stakes — exactly the cross-correlating-many-signals failure mode small quantized models hit. Keep all statistics SAS/R-only and escalate the integrative narrative to a larger model. *SAS/R owns:* The validated/frozen stats engine computes all outlier statistics, digit-preference tests, variance/correlation anomalies and date-implausibility flags; SAS/R clusters findings deterministically by site and supplies only the computed ranking. The model never computes or confirms a statistic.

✗ Safety signal triage (AE/SAE volume, lab shifts)

Binding constraint is the highest-stakes narrative class in this area — clinical-safety judgement on unblinded data where a small model's hallucination or mis-prioritization is directly patient-relevant. Keep the SLM off it; this is Beyond on-device. *SAS/R owns:* The validated safety DB/EDC computes all AE/SAE volumes and lab-shift numbers; SAS/R assembles the flagged inputs, enforces that no number originates in any model, and routes to the safety physician. The model is kept entirely off the causality and number paths.

✗ DSMB / safety-review packet assembly

Binding constraint is faithful long-form synthesis over an unblinded safety packet — the long-summary + high-stakes territory where small models drop or fabricate findings. Keep collation and TLF assembly deterministic in SAS/R; the cover narrative is Beyond on-device and must stay on-prem if escalated. *SAS/R owns:* Validated tools produce every TLF; SAS/R pulls and orders the validated tables/listings/figures, assembles the packet shell deterministically, and keeps all numbers out of any model's hands. The packet structure is templated by SAS/R, not narrated into existence by the SLM.

✗ Monitoring-visit report (MVR) summarization

Binding constraint is faithful long-document summarization plus cross-MVR theme synthesis — small quantized models drop or fabricate findings well before their advertised context limit, and the portfolio rollup is multi-document synthesis. As scoped (full MVR + cross-site rollup) this is Beyond; only a narrow single-MVR schema extraction would be Guardrails. *SAS/R owns:* SAS/R supplies de-identified MVR text, enforces the output schema on any extraction, and stores drafts pending CRA/clinical-lead confirmation of findings of record. SAS/R, not the model, does any cross-report aggregation of themes.

✗ Lessons-learned & continuous-improvement knowledge capture

Binding constraint is multi-document synthesis: clustering short records is feasible but cross-artifact thematic synthesis needs frontier-level reasoning, so escalate the synthesis step to a larger local/cloud model (de-identified) while SAS/R does retrieval and grouping and a QA/process owner curates before anything becomes guidance. *SAS/R owns:* Owns retrieval/RAG indexing, embedding-based similarity grouping of issue records, trend statistics, surfacing prior lessons at study start-up, writes, and audit log.

The pattern: a small model is safe exactly where the task is short, bounded, and structured. The moment it must reason across documents, hold long context, or write nuanced narrative, it is the wrong tool — that is not a deployment detail to tune around, it is the capability ceiling.

The SAS/R × SLM glue — `%slm_*` macros & R functions

A small helper library so calling the local model is a few lines: SAS/R sends the prompt to the offline endpoint, forces **schema-constrained JSON**, gets the structured answer, and a **SAS/R validator** checks every field against an allowlist before anything is shown to a human. The model is pinned (name + quantization + digest + temperature 0 + seed) so a run is reproducible.

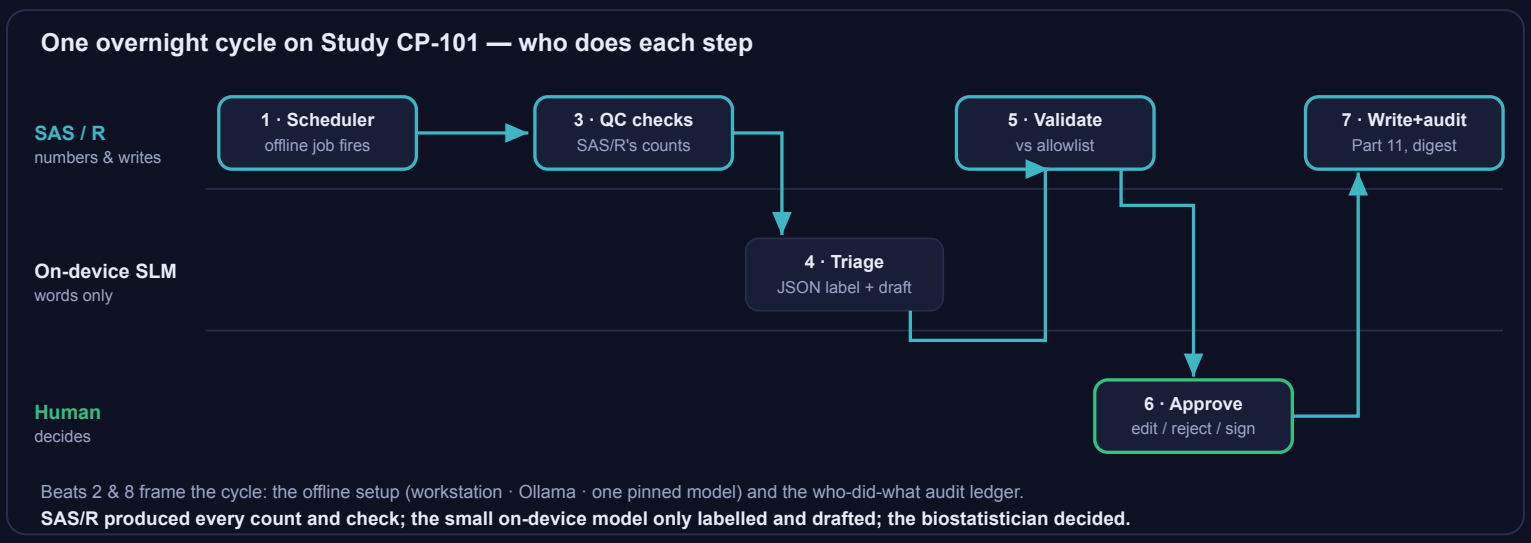
⤓ slm_macros.sas ⤓ slm_companion.R ⤓ triage_driver.sas ⤓ slm_config.sas

⤓ README (PDF)

> **Five guarantees:** offline (loopback-only, asserted in code) · schema-constrained output · SAS/R-validated against an allowlist (never trust free text) · the model pinned & frozen for reproducibility · a human gate before any write. The model never produces a number.

A morning of on-device triage — one offline workstation

A scheduled SAS/R QC run plus a small local model doing only the language layer, fully offline: SAS/R computes the findings, the model labels and drafts, SAS/R validates, the biostatistician approves, SAS/R writes and audits.

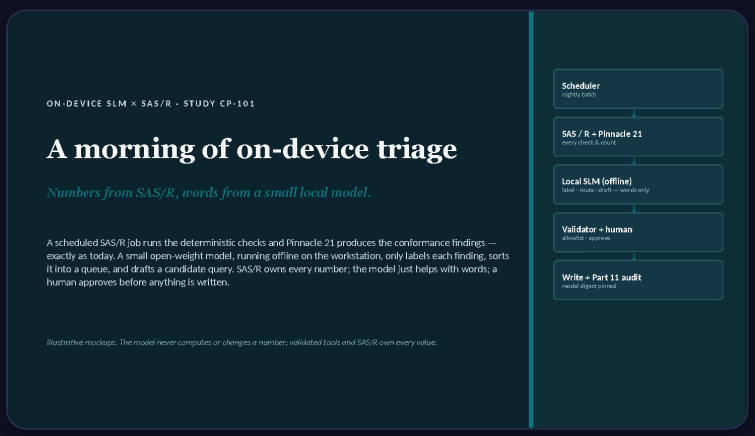


The morning in three lanes: SAS/R fires the offline job, runs every check, then validates and writes with a Part 11 trail; the on-device model does one triage-and-draft step in JSON; and the biostatistician approves each candidate before anything becomes a record — numbers from SAS/R, words from the model, the decision from the human.

▶ Play the narrated walkthrough

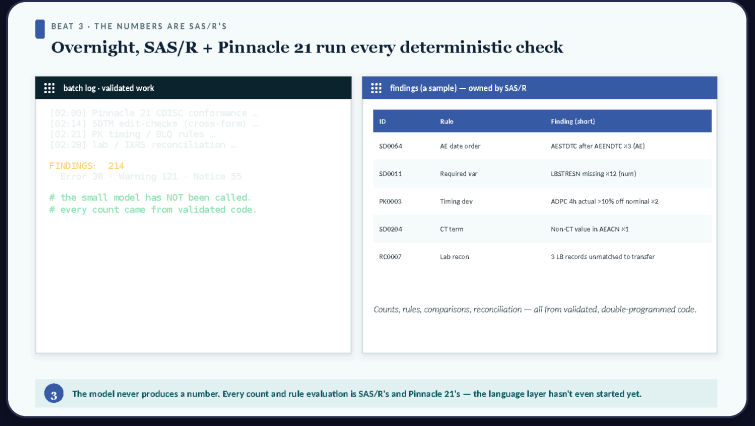
⤓ Step-by-step picture guide (PDF)

⤓ IT enablement runbook (PDF)

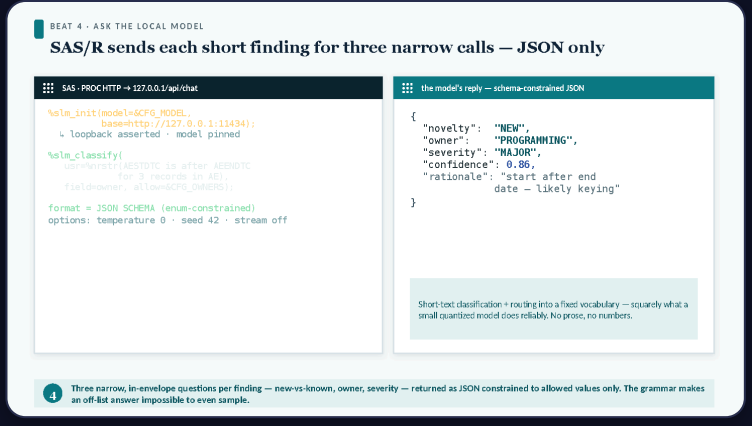


Beat 1 · The loop: numbers from SAS/R, words from a small local model — One overnight cycle on Study CP-101: a scheduled SAS/R job does every deterministic check; a small on-device model only labels, classifies, and drafts; the biostatistician approves before anything is written.

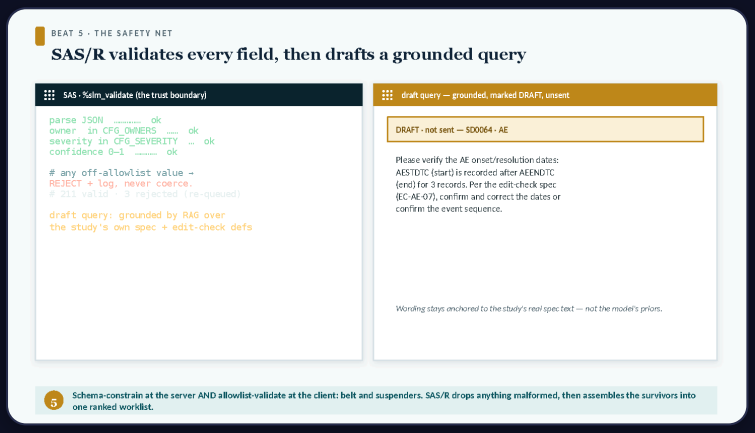
Beat 2 · The offline setup: workstation, Ollama, one pinned model — An ordinary validated workstation runs Ollama serving a pinned, quantized 7B model on 127.0.0.1. No GPU farm, no cloud account, no telemetry — the box is offline and the network cable is unplugged.



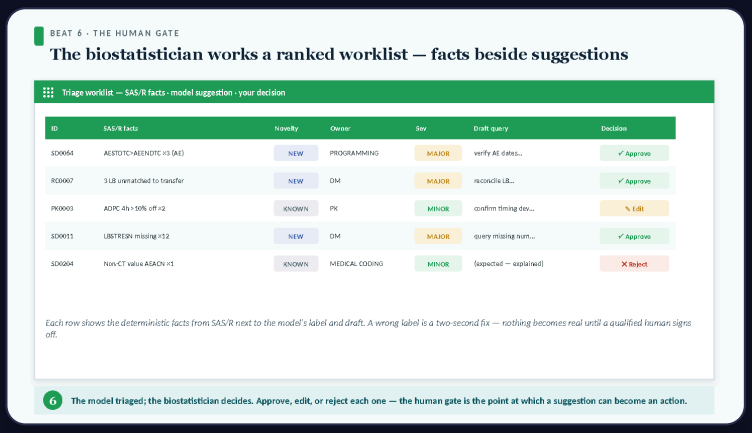
Beat 3 · SAS/R runs the deterministic checks — the counts are SAS/R's — Overnight the scheduled job runs the Pinnacle 21 CDISC conformance run plus the study's own edit checks and reconciliation; it produces 214 findings, every count and rule owned by validated code, not the model.



Beat 4 · SAS/R calls the local model to triage — JSON out, nothing else — Via PROC HTTP / httr2 to 127.0.0.1, SAS/R sends each finding's short text to the model for three narrow decisions — new-vs-known, type/severity, owner/queue — returned as schema-constrained JSON.



Beat 5 · Validator + RAG-grounded draft — the safety net around the model — SAS/R checks every returned field against an allowlist and drops anything malformed; for valid findings the model drafts a candidate query grounded by RAG over the study's own specs, marked DRAFT and unsent.



Beat 6 · The human gate: the biostatistician approves each draft — The biostatistician works the ranked worklist, sees each finding's SAS/R facts beside the model's suggested label and draft text, and approves, edits, or rejects — nothing proceeds without a human decision.

Illustrative mockups. Throughout: **SAS/R produced every count and check; the small on-device model only labelled, classified, and drafted; the biostatistician decided.**

Governance & honest limits

The governance is the same as every companion in this program; only the model changes. The line that does not move: **SAS/R and the validated engines own every number and every record of truth. The small on-device model never produces a value, never reconciles, never decides.** It classifies, extracts, routes, and drafts words — and a human approves before anything is written. Swapping cloud Claude for a small (roughly 1–9B-parameter), quantized model on a workstation tightens the data-egress story; it does not loosen the controls.

> **The red line is unchanged.** Pinnacle 21, Phoenix WinNonlin, and the EDC/CTMS own every reported and regulated value. SAS/R owns all data access, every count, check, reconciliation, and statistic, plus the validator, the writes, and the audit log. The on-device model is a constrained language sidecar SAS/R calls (PROC HTTP / httr2 → a local loopback endpoint at 127.0.0.1, served by Ollama or llama.cpp) for the narrow language sub-task only — its output is schema-validated against an allowlist and gated by a human before it touches a record.

What governs the small model

- **SAS/R owns the truth.** Every number, check, reconciliation, and stat is deterministic SAS/R; every reported/regulated value comes from a validated engine. The model is never in the number path.
- **The model only handles words.** Its job is bounded: label into a fixed bucket set, extract a field from a short passage, tag an owner/queue/severity, group near-duplicates, or draft a short templated text grounded in retrieved spec. Nothing more.
- **Constrained output, then validated.** The model is forced toward well-formed JSON by grammar-constrained decoding (llama.cpp GBNF / JSON-schema, or Ollama structured output), which masks tokens that violate the schema during sampling. This is not a guarantee — constrained decoding can still emit a truncated or incomplete object if generation runs out of tokens — so a deterministic SAS/R validator independently parses the output, checks every field against the allowed values, and drops or flags anything malformed, incomplete, or out-of-set. Free text is never trusted as-is.
- **Human gate before any write.** Every model-touched item (a label, a route, a draft query) lands in an approval queue. A biostatistician approves; only then does SAS/R write and audit.
- **Pinned, version-controlled model.** Pin the model name + quantization + weight digest, fix the decoding policy (greedy / temperature 0), and version the prompt and the RAG sources. That tuple is a frozen, re-runnable configuration — the qualified instrument is defined exactly and cannot change underneath you without the digest changing. Note the honest caveat below: pinning makes the *configuration* reproducible, not necessarily the token-for-token *output*.
- **Part 11 audit trail.** SAS/R logs the prompt, the pinned model digest, the model output, the validator result, and the human decision (who, when, approve/reject) for every item. The decision record is inspectable end-to-end.
- **Local-only, network-isolated.** The model and runtime live on the workstation; the endpoint is loopback (127.0.0.1) and the box is held on an isolated/air-gapped network by configuration and policy. This is an enforced, verifiable control, not a law of physics — the machine still has hardware interfaces, so the control is “no outbound path is configured or permitted, and that is auditable,” not “egress is impossible.” It is still the simplest data-governance story of any AI option in the program: there is no cloud endpoint to trust, no BAA to negotiate, and no egress firewall to maintain.

Roles, stated plainly

Layer	Owns	Never does

Validated engines (Pinnacle 21, Phoenix WinNonlin, EDC/CTMS)	Every reported and regulated value.	—
SAS/R	Data access, all computation/checks/reconciliation/stats, scheduling, the output validator, every write, the audit log.	Delegate any number to the model.
On-device SLM	The narrow language sub-task only — classify, extract, route, draft — as schema-constrained JSON.	Produce a number, reconcile, synthesize across documents, or make a clinical/regulatory call.
Human	Approving (or rejecting) every model-touched item before it is written; all interpretation and judgement.	Be bypassed.

A note on the model licences

“Open-weight” is not the same as OSI “open-source,” and the difference matters to legal and QA. Confirm the licence per model before qualification: Qwen3, Gemma 4, Phi-4-mini (MIT), and IBM Granite 4.1 ship under genuinely permissive (Apache 2.0 / MIT) terms, whereas Llama 3.3/3.2 carries the Meta Llama Community License (custom, with an acceptable-use policy and a >700M-MAU clause) — redistributable for our use but *not* OSI-approved, with flow-down/use restrictions. None of this changes the data-governance story (the weights run locally either way); it is a procurement and IP checklist item, not a control on the data.

Validate before production

Treat the pinned model as a qualified analytical instrument, not software you trust on faith. Risk-based GAMP 5 qualification: installation qualification on the runtime, operational qualification of the pinned model against a fixed test set (does the constrained output hold; does the validator catch the bad cases), and performance qualification on the real workflow with the human gate live. **Validate against a sandbox first; promote to production only after the pinned-model configuration passes on representative study data.** Re-qualify on any change to the model, quantization, runtime, decoding policy, or prompt — the digest or configuration changes, so the qualified state changes.

Two honest limits.
1. Capability. A small on-device model is a **narrow language helper, not a frontier brain**. It will not reliably synthesize across documents, reason over long context (small quantized models degrade well before their advertised window), or write nuanced clinical, safety, or regulatory narrative — and it hallucinates more, and is more prompt- and format-sensitive, than a large model. Work that genuinely needs that reasoning stays **deterministic SAS/R** or **escalates to a larger local or cloud model** under its own governance — it does not get forced onto a quantized small model with optimistic prompting.
2. Reproducibility. Do not claim bit-identical model output as a free property of “temperature 0 + a seed.” At temperature 0 decoding is greedy, so the seed is irrelevant; and on GPU backends (CUDA/ROCm/Metal) the same input can still yield different tokens run-to-run because batching, reduction order, and KV-cache offload are not deterministic. llama.cpp is reproducible on CPU, and a recent opt-in CUDA deterministic mode exists, but these are conditions you must pin and qualify, not assumptions. This is why the design keeps the SLM *out* of every number and behind a human gate: the regulatory reproducibility that matters lives in deterministic SAS/R and the validated engines, where it is guaranteed — the SLM’s output is always re-checked by the validator and approved by a person, so the audit trail, not the model’s determinism, is what holds.

This is necessary discipline, not a limitation to engineer around: scope the model to short, bounded, structured language tasks, keep every number in SAS/R, and keep the human in the loop.

FAQ

Is a small AI model safe to use in regulated work? Only in the narrow shape we deploy it: the on-device model never touches a record of truth, never produces a number, and never decides anything. SAS/R does the deterministic work and writes the audit log; the model emits schema-constrained output that a SAS/R validator checks against an allowlist; a human approves before anything is committed. Inside that envelope — short classification, extraction from a short passage, routing, near-duplicate grouping, templated drafting grounded in retrieved text — residual risk is low but not zero, so the human gate stays. Outside it, the model is the wrong tool, which is why 15 of the 75 components are marked *Beyond*.

Does the small model touch any of the numbers? No. SAS/R owns every count, check, reconciliation, and statistic, and reported or regulated values come only from validated engines — Pinnacle 21, Phoenix WinNonlin, the EDC/CTMS. A small quantized model is the *least* trustworthy thing in the building for anything quantitative; it hallucinates more than a frontier model and is more sensitive to prompt and format. So it is fenced out of arithmetic entirely. It labels, extracts short fields, groups duplicates, and drafts a query or note for a human to confirm — nothing else.

How is this different from the cloud-Claude hybrid? Same division of labor, very different capability and sovereignty trade. The hybrid sends de-identified language work to a frontier model that *can* reason across documents, hold long context, and write nuanced narrative — so its *Beyond* list is short. This option uses a 1–9B model that runs offline on hardware you already own, so the data-residency story is the simplest of any option, but the capability ceiling is real: the work the hybrid sends to Claude is exactly the work a small model can't do reliably. A small quantized model also degrades well before its advertised context window, so “it has a long context” is not the same as “it reasons well over one.” Pick on-device where sovereignty and reproducibility dominate and the language task is genuinely small; escalate to the hybrid when the language work needs real reasoning.

And how is it different from the M365 Copilot option? Copilot is a cloud service with no on-prem model and no model freeze — good for de-identified ops, but it can't keep unblinded or participant-level text on the box and can't be pinned for re-runnable reproducibility. The on-device SLM inverts that: it can run on a network-isolated box (loopback-only client config, plus no egress route at the infrastructure layer), and it is straightforward to pin and freeze, at the cost of the model's reasoning power. Different ends of the same spectrum — Copilot trades sovereignty for capability; the local SLM trades capability for sovereignty.

What hardware do we actually need? Less than people expect. A 3–8B model at 4–8 bit quantization (GGUF) runs on an ordinary workstation or laptop — CPU works, a single consumer GPU is faster — served locally by Ollama, llama.cpp, or LM Studio. No server farm, no cloud account. The constraint is throughput, not feasibility: keep the prompts short and the batches sized to the box. If a task needs a model big enough to need real GPU infrastructure, that is a signal the task is *Beyond* a small model in the first place.

Is it validated and reproducible? The pipeline is built to be, with an honest definition of “reproducible.” We pin the model name, quantization, and digest; set decoding to temperature 0 with a fixed seed; fix the context length and other decode options; and freeze the prompt — together a re-runnable artifact. Output is deterministic only *within a pinned environment*: the same runtime build, the same hardware and backend, the same context settings. Across different builds or GPUs, kernel and batching differences can still shift output, so we don't claim bitwise reproducibility on arbitrary hardware. That is fine, because the regulatory burden does not rest on the model: under GAMP-5 and 21 CFR Part 11, validation lives in the SAS/R validator, the allowlist, the audit trail, and the human gate. The model is a frozen, constrained language sidecar, not a validated instrument and never treated as one.

What happens to the data? It stays on the workstation. SAS/R calls the model at a local 127.0.0.1 endpoint (PROC HTTP / httr2), and the box runs with no telemetry and no egress — nothing leaves. The loopback-only client config is necessary but not sufficient on its own; the real control is that the box has no route off the network, which is an infrastructure control we assert and inspect, not just a line of code. That is the whole point of the on-device choice:

the data-sovereignty control collapses from “firewall the cloud egress” to “the box stays offline.” PHI, unblinded, and participant-level text are safe here precisely because there is nowhere for them to go.

Which model should we start with? A current 7–9B instruct model at 4–8 bit, with a 3B fallback for the lightest classification tasks. Check the licence before you commit, because “open-weight” is not “open-source”: Qwen3, Gemma 4 and IBM Granite 4.1 8B ship under Apache 2.0, while Llama 3.3/3.2 (Meta Community License + Acceptable Use Policy, MAU cap, flow-down terms) carries a custom restrictive licence that legal should clear for our use. Bigger-is-better is the correct instinct for the language work: don't reach for a 1.5B model to save memory if an 8B one clears the task. Pick one, pin model+quantization+digest, validate it against the allowlist on your own examples, and freeze it. The exact name matters less than locking it down once it works.

Walkthrough transcript

The complete narration of the “morning of on-device triage” video, as readable text.

1. Part 1

This is one overnight cycle on Study CP-101, a Phase 1 clin-pharm study. A scheduled SAS or R job runs the deterministic checks and Pinnacle 21 produces the conformance findings, exactly as they do today. The only thing that is new is a small open-weight model running locally on the workstation, called by the job to handle the language layer — labeling each finding, sorting it into a queue, and drafting a candidate query. SAS/R owns every number; the model only helps with words; and a human approves before anything touches a record of truth.

2. Part 2

The setup is deliberately boring. An ordinary workstation you already own runs Ollama, serving one quantized model — say Qwen2.5 7B at Q4_K_M, Apache-2.0 licensed — on the loopback address 127.0.0.1. The model name, quantization, and digest are pinned and registered, decoding is fixed at temperature zero with a set seed, so the same input gives the same output every run. The box stays offline: there is no cloud account, no telemetry, and the network cable is unplugged — the data-egress story is simply that nothing can leave.

3. Part 3

Overnight, the scheduled job runs the validated work. Pinnacle 21 executes the CDISC conformance run over SDTM and Define-XML, and the study's own SAS or R programs run the cross-form edit checks, the PK timing and BLQ rules, and the lab and IXRS reconciliation. This morning it produces two hundred fourteen findings. Every one of those counts, comparisons, and rule evaluations came from validated, double-programmed code — the small model has not been called yet and will never compute a number.

4. Part 4

Now SAS/R calls the local model, one finding at a time, using PROC HTTP or httr2 to the loopback endpoint. For each short finding it asks three narrow, in-envelope questions: is this new or a known recurring issue, what type and severity bucket does it fall in, and which owner or queue should it route to. The model must answer as JSON constrained to a fixed schema with allowed values only — no free prose, no numbers. This is short-text classification and routing, squarely in what a small quantized model can do reliably.

5. Part 5

Before trusting a single word, the SAS/R validator parses the JSON and checks every field against the allowed enum — any malformed or out-of-vocabulary answer is dropped and logged, never guessed. For findings that pass, the model drafts a one-line candidate query, but only grounded by retrieval over the study's own specifications and edit-check definitions, so the wording stays anchored to real source text. Every draft is labeled DRAFT and is unsent; SAS/R then assembles the survivors into a single ranked worklist, ordered by severity and aging.

6. Part 6

At the workstation the biostatistician opens the ranked worklist. Each row shows the deterministic facts from SAS/R — the rule, the records, the counts — next to the model's suggested label and its draft query text. The reviewer approves, edits, or rejects each one; a wrong label is a two-second fix, not a silent error. Nothing the model produced becomes real until a qualified human has signed off on it.

7. Part 7

Only the approved items get written. SAS/R issues the queries to the EDC and writes a 21 CFR Part 11 audit line for each one, recording the prompt, the pinned model name, quantization and digest, the draft text, and who approved it and when. Because the model is frozen and decoding is fixed, that audit line is a re-runnable artifact an inspector can reproduce. And the offline guarantee is physical, not a policy promise — the cable is unplugged, so the only thing that ever left the box was a query you approved into your own EDC.

8. Part 8

Step back and the division of labor is clean. SAS/R and Pinnacle 21 owned the data, every check and count, the validator, every write, and the audit trail. The small local model did one narrow language job — sorting findings and drafting candidate words — and it never touched a number or a record on its own. The biostatistician made every decision that mattered. That is the honest shape of an on-device win: the model saved triage time on a narrow, validated task, and the truth stayed exactly where it always was.